

FACULTAD DE CIENCIAS
GRADO EN MATEMÁTICAS
TRABAJO FIN DE GRADO
CURSO ACADÉMICO 2024-2025

TÍTULO:

**OPTIMIZACIÓN DE LARGE LANGUAGE MODELS PARA LA
REALIZACIÓN DE TAREAS**

AUTOR:

DANIEL SERRANO ALZAMORA

Resumen

Este trabajo aborda la optimización de modelos de lenguaje de gran tamaño (LLMs) mediante técnicas de ajuste eficiente, con el objetivo de adaptarlos a tareas de clasificación binaria sin reentrenar todos sus parámetros. Para ello, se combinan fundamentos teóricos del aprendizaje profundo con un estudio experimental sobre datos reales del sector bancario.

En la parte teórica, se analizan los principios del preentrenamiento auto-supervisado en LLMs, los paradigmas de modelado causal y enmascarado (con énfasis en el enfoque *span corruption* de T5), las arquitecturas *encoder-decoder* y el proceso de retropropagación. A continuación, se introducen los principales retos de optimización en LLMs, como el coste computacional y el sobreajuste, así como técnicas clásicas como la cuantización y el *model pruning*.

El eje central del trabajo es el ajuste eficiente mediante *Parameter-Efficient Fine-Tuning* (PEFT), destacando estrategias como *Adapters*, *Prefix Tuning* y, especialmente, LoRA. Esta última se describe en detalle, mostrando cómo la inserción de matrices de bajo rango en las proyecciones del modelo permite adaptar su comportamiento sin modificar los parámetros originales. Se incluye un ejemplo ilustrativo y un análisis de sus principales hiperparámetros.

En la parte práctica, se implementa un pipeline en **Python** con validación cruzada temporal y métricas estándar. Se comparan cinco modelos clásicos (Regresión Logística, Árbol de Decisión, *Random Forest*, *XGBoost* y Red Neuronal) con un LLM (**Flan-T5 base**) y su versión ajustada con LoRA.

Los resultados muestran que los modelos clásicos, tras el ajuste, ofrecen un rendimiento sólido. El LLM sin adaptar presenta un comportamiento sesgado, mientras que su versión ajustada con LoRA mejora sustancialmente su rendimiento, logrando predicciones más equilibradas y consistentes.

En conclusión, LoRA permite adaptar LLMs de forma eficiente sin requerir *hardware* especializado, demostrando ser competitivo frente a los modelos tradicionales. Como líneas futuras, se propone comparar LoRA con otras técnicas PEFT y explorar arquitecturas híbridas que combinen LLMs con modelos clásicos.

Palabras clave: Redes Neuronales, *Large Language Models* (LLMs), *Low-Rank Adaptation* (LoRA), *Parameter-Efficient Fine-Tuning* (PEFT), *Machine Learning*.

Abstract

This work addresses the optimization of Large Language Models (LLMs) through efficient fine-tuning techniques, with the aim of adapting them to binary classification tasks without retraining all their parameters. To achieve this, theoretical foundations of deep learning are combined with an experimental study based on real-world banking sector data.

The theoretical part analyzes the principles of self-supervised pretraining in LLMs, the paradigms of causal and masked language modeling (with an emphasis on the span corruption approach used in T5), encoder-decoder architectures, and the backpropagation process. It also introduces key optimization challenges in LLMs, such as computational cost and overfitting, as well as classical techniques like quantization and model pruning.

The core of this work is the efficient adaptation of large models through Parameter-Efficient Fine-Tuning (PEFT), highlighting strategies such as Adapters, Prefix Tuning, and, in particular, LoRA. The latter is described in detail, showing how the insertion of low-rank matrices into the model's projections allows its behavior to be adapted without modifying the original parameters. An illustrative example and an analysis of its main hyperparameters are included.

On the practical side, a `Python` pipeline is implemented using temporal cross-validation and standard metrics. Five classical models (Logistic Regression, Decision Tree, Random Forest, XGBoost, and Neural Network) are compared with an LLM (`Flan-T5 base`) and its LoRA-adjusted version.

The results show that classical models, after hyperparameter tuning, deliver solid performance. The unadapted LLM exhibits biased behavior, whereas the LoRA-adjusted version significantly improves its performance, achieving more balanced and consistent predictions.

In conclusion, LoRA enables efficient adaptation of LLMs without the need for specialized hardware, proving competitive with traditional models. As future work, it is proposed to compare LoRA with other PEFT techniques and to explore hybrid architectures that combine LLMs with classical models.

Keywords: Neural Networks, Large Language Models (LLMs), Low-Rank Adaptation (LoRA), Parameter-Efficient Fine-Tuning (PEFT), Machine Learning.

Índice

1. Introducción	6
2. Preliminares - Redes Neuronales	7
3. <i>Large Language Models</i> (LLMs)	11
3.1. Tokenización y representaciones vectoriales	12
3.1.1. Algoritmos de tokenización	12
3.1.2. Procesamiento post-tokenización	14
3.1.3. Codificación posicional	16
3.2. Preentrenamiento de LLMs	16
3.2.1. Modelado de Lenguaje Causal (CLM)	17
3.2.2. Modelado de Lenguaje Enmascarado (MLM)	17
3.3. Arquitectura de los LLMs	19
3.4. Salida del modelo	21
4. Optimización en LLMs	23
4.1. Principales retos y soluciones de optimización	23
4.2. Técnicas clásicas de optimización	24
4.3. Optimización del proceso de entrenamiento	25
4.4. <i>Fine-Tuning</i> de modelos	26
4.5. <i>Parameter-Efficient Fine-Tuning</i> (PEFT)	26
5. Técnica <i>Low-Rank Adaptation</i> (LoRA)	28
5.1. Formulación matemática y eficiencia de LoRA	28
5.2. Hiperparámetros fundamentales en LoRA	33
5.3. Aplicación en arquitecturas <i>Transformer</i>	34
5.4. Casos de uso y eficacia empírica	35
6. Diseño y desarrollo del caso práctico	35
6.1. Descripción del conjunto de datos	36

6.2. Evaluación de modelos y estrategias de validación	36
6.3. Modelos de <i>Machine Learning</i>	42
6.4. Modelos LLM y adaptación con LoRA	44
6.4.1. Modelo LLM sin ajuste	45
6.4.2. Modelo LLM ajustado con LoRA	46
6.5. Resultados y comparativa entre modelos	48
6.5.1. Comparación en entrenamiento, validación y test de modelos clásicos	48
6.5.2. Comparación entre LLM y LLM ajustado con LoRA	52
6.5.3. Comparación final	53
7. Conclusiones	54
8. Trabajos futuros	56
Referencias	58
A. Hiperparámetros ajustables en LoRa y en el entrenamiento	62
A.1. Hiperparámetros de LoraConfig	62
A.2. Hiperparámetros de TrainingArguments	63
B. Tiempos de entrenamiento y evaluación LLM y LoRA	64
C. Detalles del desarrollo del trabajo	64

1. Introducción

En los últimos años, los modelos de lenguaje de gran tamaño (*Large Language Models*, LLMs) se han consolidado como una de las herramientas más influyentes de la inteligencia artificial moderna, gracias a su capacidad para comprender y generar texto de forma coherente y contextualizada. Su impacto se ha extendido a numerosos sectores, desde los asistentes virtuales y la redacción automática hasta la programación y el análisis de datos, posicionándose como elementos clave en el avance tecnológico contemporáneo.

No obstante, el despliegue de estos modelos plantea importantes desafíos técnicos y prácticos. Por un lado, su gran tamaño dificulta su ejecución en dispositivos con recursos limitados, como GPUs o RAM convencionales, lo que ha impulsado el desarrollo de técnicas como la cuantización o la poda (*pruning*), orientadas a reducir la huella computacional sin comprometer sustancialmente el rendimiento. Por otro, en muchos contextos resulta inviable reentrenar un modelo de gran escala desde cero para cada nueva tarea, lo que ha motivado la investigación de métodos que permitan adaptar modelos preentrenados de forma eficiente, reduciendo así los costes computacionales y fomentando su democratización.

Entre estas técnicas, destaca especialmente la de *Low-Rank Adaptation* (LoRA), que permite especializar modelos preentrenados generalistas mediante la inserción de matrices entrenables de bajo rango, en lugar de ajustar todos sus parámetros. De este modo, se transforma un único modelo base en múltiples versiones adaptadas a tareas concretas, manteniendo su estructura intacta y reduciendo significativamente el coste computacional. Esta estrategia contrasta con el desarrollo de LLMs generalistas diseñados explícitamente para abordar tareas frecuentes —como completar frases, resumir textos o responder preguntas—, ya que LoRA permite adaptar con precisión un modelo existente a las necesidades específicas de una tarea sin necesidad de entrenar un modelo desde cero.

Este Trabajo de Fin de Grado se enmarca en dicho contexto y tiene como objetivo principal estudiar cómo optimizar el comportamiento de los LLMs en tareas específicas mediante técnicas de adaptación eficiente. Para ello, se propone:

- Analizar los fundamentos matemáticos y computacionales que sustentan el funcionamiento de los LLMs y las técnicas de *fine-tuning* eficientes, con especial atención a LoRA.
- Examinar los principales desafíos que presentan estos modelos en términos de escalabilidad, consumo de recursos y adaptabilidad.

- Comparar empíricamente el rendimiento de modelos LLM adaptados con LoRA frente a algoritmos clásicos de *machine learning*, evaluando su precisión, eficiencia y capacidad de generalización en tareas específicas de predicción o clasificación.

La finalidad última de este trabajo es ofrecer una visión crítica, rigurosa y fundamentada sobre el potencial real de los LLMs optimizados, evaluando su aplicabilidad en contextos reales desde una perspectiva tanto teórica como práctica. Para ello, se utilizarán herramientas de desarrollo en **Python** y se aplicarán métricas de evaluación habituales en tareas de clasificación, tales como *accuracy*, *precision*, *recall* y *F1-Score*.

2. Preliminares - Redes Neuronales

Las redes neuronales artificiales (*Artificial Neural Networks*, ANN) constituyen una clase de modelos computacionales inspirados en la estructura y el funcionamiento del cerebro humano. Estas redes han demostrado ser herramientas extremadamente eficaces en tareas de clasificación, regresión, procesamiento de imágenes y análisis de lenguaje natural, entre otras. Su capacidad de aproximar funciones no lineales las convierte en elementos fundamentales del aprendizaje automático moderno, y su evolución ha dado lugar a arquitecturas más sofisticadas, como los modelos de lenguaje de gran escala (LLMs).

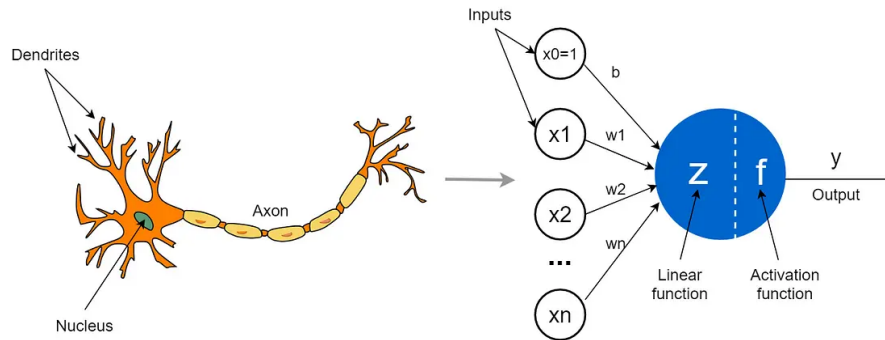


Figura 1: Comparación entre una neurona biológica y una neurona artificial.

Modelo de neurona artificial (Perceptrón)

La unidad básica de una red neuronal es la **neurona artificial** o **perceptrón**. Dado un vector de entrada $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$ y un vector de pesos asociados $\mathbf{w} =$

$(w_1, w_2, \dots, w_n) \in \mathbb{R}^n$, la neurona calcula una combinación lineal de las entradas y le añade un sesgo $b \in \mathbb{R}$. Este resultado se denomina **potencial de activación**:

$$z = \mathbf{w} \cdot \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b \quad (1)$$

A continuación, se aplica una **función de activación** $\phi(z)$, que introduce no linealidad en el modelo y permite a la red neuronal aproximar funciones arbitrariamente complejas. Así, la salida de la neurona es:

$$a = \phi(z) = \phi\left(\sum_{i=1}^n w_i x_i + b\right) \quad (2)$$

Entre las funciones de activación más habituales se encuentran:

- **Función sigmoide:** $\phi(z) = \frac{1}{1 + e^{-z}}$
- **Tangente hiperbólica:** $\phi(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- **ReLU (*Rectified Linear Unit*):** $\phi(z) = \max(0, z)$

Arquitecturas de redes neuronales

Una red neuronal está compuesta por múltiples neuronas organizadas en capas. Estas arquitecturas se clasifican comúnmente como:

- **Redes neuronales *feedforward* (FNN):** La información fluye en una única dirección desde la capa de entrada hacia la capa de salida, sin ciclos.
- **Redes neuronales profundas (DNN):** Redes *feedforward* con múltiples capas ocultas, lo que incrementa la capacidad de representación del modelo. La profundidad hace referencia al número de capas.
- **Redes neuronales recurrentes (RNN):** Permiten conexiones cíclicas, lo que las hace adecuadas para datos secuenciales como texto o series temporales. La salida depende no solo de la entrada actual, sino también del estado anterior.
- **Redes convolucionales (CNN):** Aunque originalmente diseñadas para procesamiento de imágenes, pueden utilizarse en NLP mediante convoluciones sobre secuencias. Son eficientes en la extracción de patrones locales.

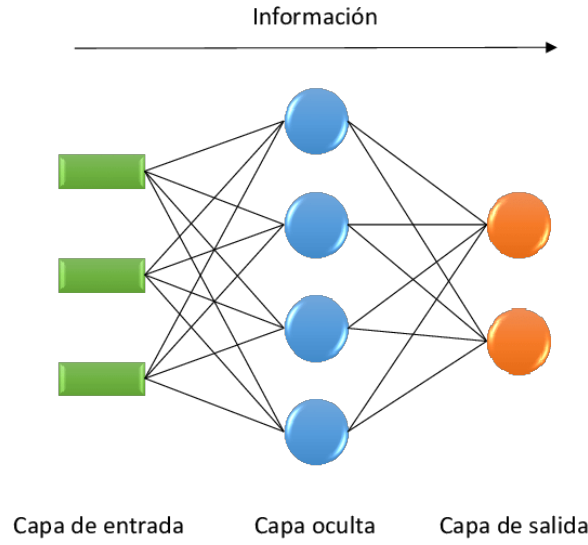


Figura 2: Ejemplo esquemático de una red neuronal.

Las redes modernas, como los *Transformers*, combinan mecanismos avanzados de atención con arquitecturas profundamente paralelizables, superando muchas de las limitaciones estructurales de las RNN.

Función de coste y optimización

El aprendizaje en una red neuronal se basa en la minimización de una función de pérdida o coste $\mathcal{L}(y, \hat{y})$, que mide la discrepancia entre la salida esperada y y la predicción del modelo $\hat{y}$. Por ejemplo:

- Para problemas de clasificación binaria: **entropía cruzada**

$$\mathcal{L}(y, \hat{y}) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}) \quad (3)$$

- Para problemas de regresión: **error cuadrático medio**

$$\mathcal{L}(y, \hat{y}) = (y - \hat{y})^2 \quad (4)$$

El objetivo es minimizar la pérdida media sobre el conjunto de entrenamiento:

$$Loss = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(y^{(i)}, \hat{y}^{(i)}) \quad (5)$$

Para ello, se emplea el algoritmo de **descenso del gradiente**, que consiste en actualizar los parámetros $(\mathbf{w}, b)$ en una dirección que reduzca el valor de la función objetivo. En particular, se elige la dirección del gradiente negativo, ya que es aquella en la que la función decrece más rápidamente en un entorno local.

Formalmente, dado un punto $\mathbf{w}$ y una función diferenciable $Loss(\mathbf{w})$, un vector $\mathbf{d}$ es una **dirección de descenso** si satisface:

$$\nabla Loss(\mathbf{w})^\top \mathbf{d} < 0 \quad (6)$$

La dirección de descenso más pronunciada es el gradiente negativo, lo que conduce a la siguiente regla de actualización:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} Loss, \quad b \leftarrow b - \eta \nabla_b Loss \quad (7)$$

donde $\eta > 0$ es la **tasa de aprendizaje**, que determina el tamaño del paso en cada iteración. Si η es muy grande, el algoritmo puede divergir; si es muy pequeña, la convergencia puede ser muy lenta.

El cálculo eficiente de los gradientes se realiza mediante el algoritmo de **retropropagación** (*backpropagation*), que aplica la **regla de la cadena** para computar las derivadas parciales de la función de pérdida con respecto a cada parámetro de la red.

Sea a_i la activación de la neurona i y z_j la entrada a la neurona j , dada por $z_j = \sum_i w_{ji} a_i + b_j$, y sea ϕ la función de activación, de modo que $a_j = \phi(z_j)$.

La derivada de la función de pérdida $\mathcal{L}$ con respecto a un peso w_{ji} se obtiene como:

$$\frac{\partial \mathcal{L}}{\partial w_{ji}} = \frac{\partial \mathcal{L}}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ji}} = \delta_j \cdot a_i \quad (8)$$

donde $\delta_j := \frac{\partial \mathcal{L}}{\partial z_j}$ representa el **error** de la neurona j .

Para la capa de salida, este error se calcula como:

$$\delta_j = \frac{\partial \mathcal{L}}{\partial a_j} \cdot \phi'(z_j) \quad (9)$$

y para capas ocultas, se propaga hacia atrás como:

$$\delta_j = \left(\sum_k w_{kj} \delta_k \right) \cdot \phi'(z_j) \quad (10)$$

donde la suma recorre todas las neuronas k a las que j conecta directamente. La derivada respecto al sesgo b_j se obtiene de forma análoga:

$$\frac{\partial \mathcal{L}}{\partial b_j} = \delta_j \quad (11)$$

Este procedimiento se repite desde la capa de salida hasta la entrada, permitiendo actualizar todos los pesos y sesgos de la red de manera eficiente, incluso en arquitecturas profundas. Constituye el núcleo del entrenamiento de redes neuronales mediante aprendizaje supervisado.

Las redes neuronales ofrecen una capacidad de modelado excepcional y son capaces de aproximar funciones altamente no lineales, lo que las convierte en herramientas muy potentes para tareas complejas de clasificación, regresión o detección de patrones. Además, pueden escalar bien con grandes volúmenes de datos y permiten el uso de arquitecturas más sofisticadas para tareas específicas.

No obstante, presentan ciertas limitaciones importantes, como la necesidad de grandes cantidades de datos y tiempo de cómputo para entrenarse adecuadamente, una alta sensibilidad a la elección de hiperparámetros, y una falta de interpretabilidad que dificulta el análisis de sus decisiones internas.

Una exposición detallada de este algoritmo y su formulación mediante derivadas parciales puede consultarse en el libro de Goodfellow, Bengio y Courville (2016) [9], concretamente en la Sección 6.5, donde se analiza el papel de la regla de la cadena en el cálculo de gradientes a través de múltiples capas.

3. *Large Language Models* (LLMs)

Los *Large Language Models* (LLMs) constituyen una clase de modelos de aprendizaje profundo entrenados para procesar y generar lenguaje natural. Estos modelos se basan en arquitecturas altamente parametrizadas, entrenadas sobre grandes corpus textuales, y logran aprender representaciones estadísticas complejas del lenguaje. Formalmente, un LLM se define como una función parametrizada:

$$f_\theta : \mathcal{X}^n \rightarrow \mathcal{Y} \quad (12)$$

donde $\mathcal{X}$ es el vocabulario de entrada, $\mathcal{X}^n$ representa una secuencia de *tokens* de longitud n , $\mathcal{Y}$ es el conjunto de *tokens* de salida (que puede coincidir con $\mathcal{X}$) y $\theta \in \mathbb{R}^d$

representa el conjunto de parámetros del modelo.

El objetivo principal de los LLMs es modelar la distribución de probabilidad de secuencias de lenguaje natural. Dado un texto de entrada $(x_1, x_2, \dots, x_{t-1})$, el modelo estima la probabilidad condicional:

$$P(x_t \mid x_1, x_2, \dots, x_{t-1}; \theta) \quad (13)$$

lo cual permite realizar tareas como predicción, generación, traducción, completado de texto, etc.

3.1. Tokenización y representaciones vectoriales

Los modelos de lenguaje no trabajan directamente con texto en lenguaje natural, sino con representaciones numéricas que puedan ser procesadas por redes neuronales. El primer paso fundamental es la **tokenización**, que consiste en dividir una secuencia de texto en unidades elementales llamadas *tokens*. Posteriormente, cada *token* se transforma en un vector denso mediante una capa de *embeddings*, lo que permite al modelo manipular semánticamente el texto.

Un *token* puede representar una palabra, una subpalabra, un carácter o incluso una secuencia especial. La elección del tipo de tokenización influye directamente en la eficiencia y generalización del modelo.

3.1.1. Algoritmos de tokenización

Los LLMs modernos emplean tokenizadores basados en subpalabras, lo cual permite reducir el tamaño del vocabulario y manejar de forma eficiente palabras no vistas. La elección del algoritmo implica compromisos (*trade-offs*) entre eficiencia, cobertura léxica y capacidad de generalización:

- **Byte Pair Encoding (BPE)** [28]: Comienza con un vocabulario basado en caracteres y fusiona iterativamente los pares de símbolos más frecuentes para formar nuevas unidades. Ventajas:
 - Eficiente para palabras raras (ej: “*quark*” $\rightarrow$ [“*qu*”, “*ark*”]).
 - Menos fragmentación que los tokenizadores por palabras completas.

Limitaciones:

- Puede subdividir morfemas lingüísticamente coherentes (ej: “*indescribable*” → [“in”, “describe”, “able”]).

Usado en modelos como GPT-2 y GPT-3¹[2], desarrollados por OpenAI², y en la familia LLaMA, desarrollada por Meta AI³. En cuanto a las versiones más recientes como GPT-4 y GPT-4.5, el tipo de tokenizador utilizado no ha sido publicado oficialmente.

- **WordPiece** [27]: Similar a BPE. Realiza una optimización basada en la probabilidad de aparición de subpalabras fusionando pares que maximizan la verosimilitud del lenguaje. Ventajas:

- Mejor manejo de morfología compleja (ej: “*transformers*” → [“transform”, “ers”]).
- Vocabulario más compacto para lenguas fusionantes.

Limitaciones:

- Requiere pre-entrenar un modelo de lenguaje inicial.

Usado en BERT y modelos derivados.

- **Unigram Language Model** [17]: Parte de un conjunto inicial de subpalabras candidatas y elimina aquellas que menos contribuyen a maximizar la probabilidad total del corpus. A diferencia de BPE y *WordPiece*, este enfoque no realiza fusiones iterativas, sino que selecciona la mejor combinación de *tokens* de forma probabilística.

Ventajas:

- Óptimo para corpus multilingües y tareas generativas.
- Permite una mayor flexibilidad en el tamaño del vocabulario.
- Produce tokenizaciones más robustas en presencia de ruido o escritura informal.

Limitaciones:

- Su entrenamiento es computacionalmente más costoso.

¹<https://github.com/openai/gpt-3>

²<https://openai.com/research>

³<https://ai.meta.com/llama/>

Este algoritmo ha sido utilizado en los modelos T5 [24] y mT5, incluyendo Flan-T5 base [3], el modelo empleado en este trabajo. Su capacidad para manejar datos heterogéneos y generar secuencias de forma fluida lo hace especialmente adecuado para el enfoque basado en transformación textual de datos tabulares adoptado en el caso práctico.

Ejemplo comparativo: La frase “*Transformers are powerful*” se tokeniza de forma distinta según el algoritmo utilizado:

- **BPE (GPT-2):** [“Transform”, “ers”, “ are”, “ powerful”]
- **WordPiece (BERT):** [“Transform”, “##ers”, “are”, “powerful”]
- **Unigram Language Model (T5):** [“Trans”, “former”, “s”, “are”, “powerful”]
(posible segmentación)

Nótese cómo **WordPiece** utiliza el prefijo “##” para indicar subpalabras no iniciales, mientras que **BPE** mantiene los espacios mediante *tokens* como “ are”. En contraste, el **Unigram Language Model** selecciona la secuencia de subpalabras que maximiza la probabilidad global de la frase, lo que puede dar lugar a segmentaciones distintas entre ejecuciones o según el vocabulario aprendido. Esto aporta flexibilidad pero introduce cierta variabilidad.

3.1.2. Procesamiento post-tokenización

Tras la tokenización, cada *token* t_i se mapea a un ID entero mediante un vocabulario predefinido ($|V| \approx 30k - 50k$). Para manejar secuencias de longitud variable:

- **Padding:** Relleno con un *token* especial [PAD] hasta una longitud máxima.
- **Truncamiento:** Corte de secuencias excesivamente largas.

Capa de *embeddings*. Cada ID entero t_i se convierte en un vector denso $x_i \in \mathbb{R}^d$ mediante una matriz de *embeddings* $E \in \mathbb{R}^{|V| \times d}$, donde $|V|$ es el tamaño del vocabulario y d la dimensión del espacio vectorial:

$$x_i = E(t_i) \tag{14}$$

Esta matriz es un parámetro del modelo que se entrena conjuntamente con el resto. Los *embeddings* permiten capturar relaciones semánticas entre *tokens*, donde elementos con

significados similares tienen representaciones vectoriales cercanas (ej.: “*king*” y “*queen*”). En los LLMs modernos, estas representaciones evolucionan a través de múltiples capas, integrando información contextual más rica.

La Figura 3 muestra una proyección bidimensional de *embeddings* de palabras preentrenadas con el modelo GloVe (50 dimensiones), obtenida mediante Análisis de Componentes Principales (PCA). Se observa cómo palabras con vínculos semánticos —como *king*, *queen*, *prince*, *princess* o *loan*, *credit*, *bank*, *money*— tienden a agruparse espacialmente, reflejando las relaciones latentes capturadas en el espacio de alta dimensión.

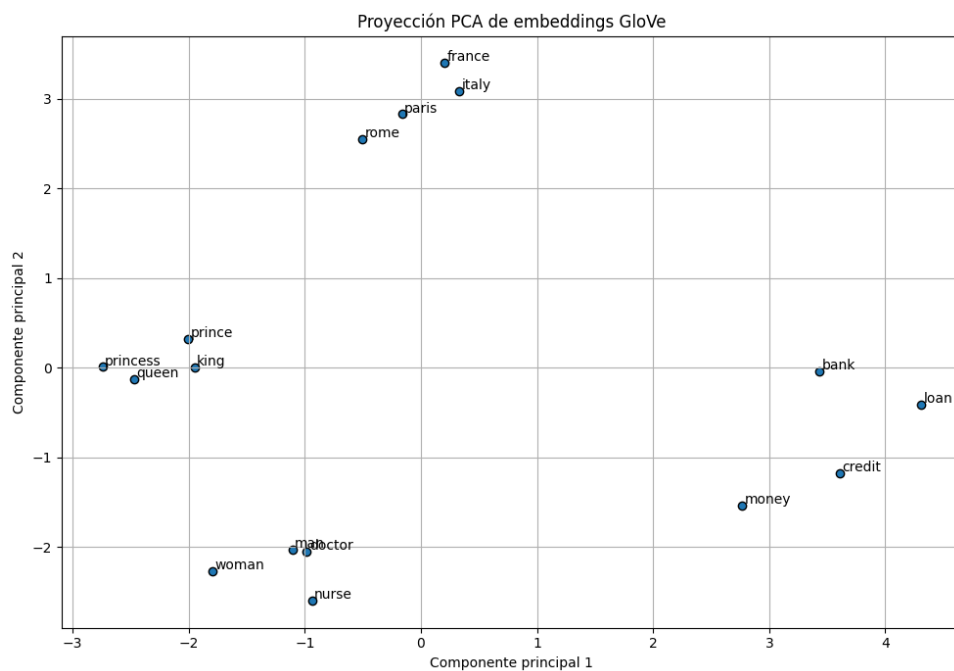


Figura 3: Proyección PCA de embeddings de palabras preentrenadas (GloVe).

Las palabras incluidas fueron seleccionadas por su relevancia temática: agrupaciones de género y nobleza, topónimos y vocabulario financiero vinculado al dominio de este trabajo. PCA se eligió como técnica de reducción por su simplicidad, velocidad y capacidad de preservar la estructura global del espacio. Aunque existen métodos alternativos como t-SNE o UMAP —más eficaces para revelar relaciones locales complejas—, en este contexto PCA proporciona una representación clara y reproducible que resulta adecuada para ilustrar la organización semántica de los *embeddings*.

3.1.3. Codificación posicional

Para inyectar información secuencial en la arquitectura *Transformer* (que es agnóstica al orden), se añaden vectores posicionales p_i a los *embeddings*:

$$z_i = x_i + p_i \quad (15)$$

Los enfoques principales son:

- **Sinusoidal**: propuesta por Vaswani et al. (2017) [31], usa funciones seno y coseno de diferentes frecuencias para generar patrones posicionales fijos. Para la posición i y dimensión j , se define:

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right), \quad p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right) \quad (16)$$

Ventaja: Generaliza a longitudes no vistas durante el entrenamiento.

- **Aprendible**: los vectores p_i se inicializan como parámetros del modelo y se optimizan durante el entrenamiento. Ventaja: Mayor flexibilidad para longitudes frecuentes en los datos.

Flujo completo de entrada. El proceso completo desde el texto hasta la entrada del *Transformer* puede esquematizarse como sigue:

$$\text{Texto} \rightarrow \text{Tokenización} \rightarrow \text{IDs} \rightarrow \underbrace{\mathbf{z}_i}_{\substack{\text{Embeddings} \\ \oplus \\ \text{Posición}}} \rightarrow \text{Input al Transformer}$$

Este proceso de representación es crucial para que el modelo capture tanto la semántica de las palabras como la estructura del lenguaje.

3.2. Preentrenamiento de LLMs

Los modelos de lenguaje de gran escala (LLMs) se entrenan inicialmente mediante un proceso de preentrenamiento auto-supervisado, que les permite aprender representaciones lingüísticas generales a partir de grandes corpus textuales. Este preentrenamiento puede seguir distintos paradigmas, cada uno asociado a un tipo de arquitectura *Transformer* y a un objetivo de aprendizaje específico.

En términos estructurales, los modelos actuales se agrupan en tres tipos principales de arquitectura:

- **Encoder-only:** emplean únicamente la parte codificadora del *Transformer*, lo que permite atención bidireccional completa. Se utilizan principalmente en tareas de comprensión.
- **Decoder-only:** utilizan solo la parte decodificadora, con atención causal unidireccional. Están orientados a tareas de generación de texto.
- **Encoder-decoder:** combinan ambos bloques y permiten tareas complejas de entrada-salida, como traducción o resumen.

Esta distinción arquitectónica es clave para entender las diferencias entre los métodos de preentrenamiento que se describen a continuación.

3.2.1. Modelado de Lenguaje Causal (CLM)

El CLM [26], también llamado **autoregresivo**, entrena al modelo para predecir el siguiente *token* en una secuencia usando únicamente el contexto precedente. La función de pérdida asociada es la entropía cruzada:

$$\mathcal{L}_{\text{CLM}}(\theta) = - \sum_{t=1}^T \log P(x_t | x_{<t}; \theta) \quad (17)$$

donde T es la longitud de la secuencia.

Este paradigma, implementado en arquitecturas *decoder-only* como GPT y LLaMA, utiliza una máscara de atención triangular que restringe cada posición a atender solo a *tokens* anteriores, garantizando coherencia en la generación pero limitando el acceso a contexto futuro. Su naturaleza incremental (*token* por *token*) lo hace especialmente adecuado para tareas de generación abierta donde la fluidez y la coherencia local son críticas [26, 29].

Ejemplo: Para la secuencia “*El gato [MASK]*”:

- Contexto disponible: [“El”, “gato”]
- El modelo predice “[MASK]” usando solo esos *tokens* (por ejemplo, “duerme”).

3.2.2. Modelado de Lenguaje Enmascarado (MLM)

El MLM, introducido por BERT [4], enmascara *tokens* aleatorios (típicamente el 15 % del texto) y los reconstruye utilizando el contexto bidireccional completo:

$$\mathcal{L}_{\text{MLM}}(\theta) = - \sum_{i \in \mathcal{M}} \log P(x_i \mid x_{\text{masked}}; \theta) \quad (18)$$

donde $\mathcal{M}$ es el conjunto de posiciones enmascaradas y x_{masked} representa la secuencia de entrada con los *tokens* en $\mathcal{M}$ sustituidos por el símbolo [MASK].

Este enfoque, al emplear arquitecturas *encoder-only* y permitir atención ilimitada en ambas direcciones, captura dependencias complejas entre todas las palabras de la secuencia, mostrando superior rendimiento en tareas de comprensión como clasificación o extracción de información [20]. Sin embargo, su naturaleza no autoregresiva lo hace menos adecuado para generación secuencial directa.

Ejemplo: Para la secuencia “El [MASK] está en el jardín”:

- Contexto disponible: [“El”, “está”, “en”, “el”, “jardín”]
- El modelo infiere “[MASK]” usando todo el contexto circundante (p.ej., “gato” o “perro”).

La elección entre CLM y MLM depende del objetivo de aplicación. Mientras CLM domina en generación de texto gracias a su coherencia autoregresiva (evidenciada por modelos como GPT-3 en tareas de completado), MLM sobresale en comprensión del lenguaje, como demostró BERT al establecer nuevos récords en *benchmarks* como GLUE [33]. Esta dicotomía refleja un compromiso entre capacidad generativa (CLM) y poder representacional (MLM), donde la arquitectura del modelo (*decoder-only* vs *encoder-only*) y el patrón de atención (autoregresivo vs bidireccional) son los factores diferenciadores clave [2].

En este trabajo se ha utilizado el modelo **Flan-T5 base**, una variante de T5 entrenada con el paradigma de *span corruption*, un enfoque de preentrenamiento basado en el enmascaramiento de fragmentos consecutivos del texto. Este enfoque puede considerarse una extensión generalizada del MLM clásico, y se enmarca dentro de los métodos *encoder-decoder* bidireccionales. A diferencia del MLM de BERT, donde se enmascaran *tokens* individuales, en T5 se enmascaran *spans* completos y se entrena al modelo para reconstruirlos como una secuencia, lo que mejora su capacidad para tareas tanto de comprensión como de generación. Este paradigma se resume en la última fila de la Tabla 1, junto con sus diferencias clave respecto a los enfoques CLM y MLM.

Paradigma	Arquitectura	Contexto de atención	Objetivo de preentrenamiento	Aplicaciones típicas	Ejemplos
Modelado de lenguaje causal (CLM)	<i>Decoder-only</i>	Unidireccional (solo pasado)	Predecir el siguiente <i>token</i>	Generación de texto	GPT, LLaMA
Modelado de lenguaje enmascarado (MLM)	<i>Encoder-only</i>	Bidireccional completo	Predecir <i>tokens</i> ocultos	Clasificación, codificación, comprensión	BERT, RoBERTa
<i>Denoising</i> / <i>Span masking</i>	<i>Encoder-Decoder</i>	Enmascaramiento de <i>spans</i> y reconstrucción secuencial	Reconstruir texto corrupto (<i>span corruption</i>)	Tareas mixtas (generación + entrada-salida supervisada)	T5, Flan-T5

Tabla 1: Comparación de paradigmas de preentrenamiento en LLMs.

3.3. Arquitectura de los LLMs

Los LLMs modernos se basan en la arquitectura de *Transformers*, introducida por Vaswani et al. en 2017 [31]. Esta arquitectura sustituye las redes recurrentes tradicionales por mecanismos de atención, permitiendo un tratamiento paralelo de los datos y una mayor capacidad para modelar dependencias de largo alcance (véase la Figura 4).

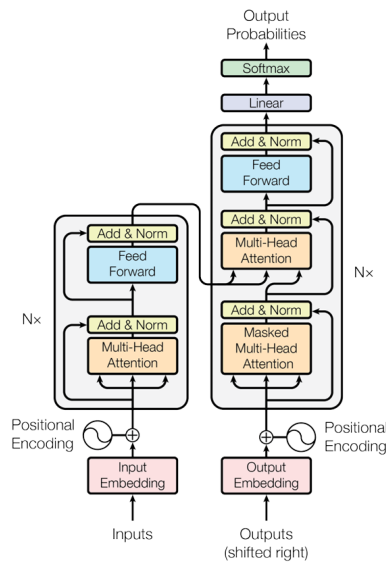


Figura 4: Arquitectura general del modelo Transformer.

En concreto, los LLMs utilizan la pila de **decoders** del transformador, compuesta por múltiples capas apiladas. Cada capa opera sobre los vectores de entrada ya transformados mediante *embeddings* y codificación posicional (véase la Sección 3.1), e incluye los siguientes componentes principales:

1. **Self-Attention**: Este mecanismo permite que cada *token* pueda atender a todos los *tokens* anteriores de la secuencia, aprendiendo representaciones contextuales. Dado un conjunto de vectores de entrada $X \in \mathbb{R}^{n \times d}$, se proyecta linealmente a tres matrices (consultas Q , claves K y valores V):

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \quad (19)$$

donde $W^Q, W^K, W^V \in \mathbb{R}^{d \times d_k}$ son matrices de parámetros aprendibles. La atención escalar multiplicativa se define como:

$$Attention(Q, K, V) = softmax \left(\frac{QK^\top}{\sqrt{d_k}} \right) V \quad (20)$$

donde d_k es la dimensión de las claves. En los *decoders* se emplea además una máscara triangular que impide que cada *token* atienda a posiciones futuras, preservando la autoregresividad del modelo.

2. **Multi-Head Attention**: Para permitir que el modelo capture múltiples tipos de relaciones contextuales simultáneamente, se utiliza un mecanismo de atención con múltiples cabezas. Cada cabeza realiza una operación de atención independiente con diferentes parámetros. Las salidas se concatenan y se proyectan mediante otra matriz aprendible:

$$MHA(Q, K, V) = Concat(head_1, \dots, head_h)W^O \quad (21)$$

donde cada *head* es una instancia de la atención escalar y $W^O \in \mathbb{R}^{hd_k \times d}$.

- **Especialización**: Cabezas individuales capturan distintos tipos de relaciones (ej: una cabeza puede enfocarse en dependencias sintácticas sujeto-verbo, mientras otra detecta relaciones semánticas campo-término) [32].
- **Máscara causal**: En *decoders*, se aplica una máscara triangular ($-\infty$ para posiciones futuras) para preservar la propiedad autoregresiva.

3. **Feed-Forward Network (FFN)**: Cada *token* se procesa de forma independiente mediante una red neuronal de dos capas lineales con una activación intermedia, normalmente ReLU o GELU:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (22)$$

donde $W_1 \in \mathbb{R}^{d \times d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times d}$, y d_{ff} suele ser mucho mayor que d , proporcionando capacidad de modelado adicional.

4. **Residual Connections y Normalización**: Para facilitar la propagación del gradiente y estabilizar el entrenamiento, cada subbloque (atención y FFN) se rodea de una conexión residual y una normalización de capa:

$$\text{LayerNorm}(x + \text{Sublayer}(x)) \quad (23)$$

LayerNorm vs *BatchNorm*: LN normaliza a lo largo de la dimensión de características ($\mathbb{R}^d$) en lugar de por *batch* ($\mathbb{R}^B$), lo que es crucial para:

- Manejar secuencias de longitud variable (*batches* con *padding*).
- Evitar inestabilidades por dependencia entre ejemplos del *batch* [1].

Esta estructura favorece una optimización más estable y una generalización mejorada.

3.4. Salida del modelo

Una vez procesada la entrada a través de todas las capas del decodificador, el modelo genera un vector de representación contextualizada $z_t \in \mathbb{R}^d$ correspondiente al *token* en la posición t . Este vector contiene la información semántica y sintáctica necesaria para predecir el siguiente *token* de la secuencia.

Para convertir esta representación en una distribución de probabilidad sobre el vocabulario, se aplica una capa lineal seguida de una función *softmax*:

$$P(x_t \mid x_1, \dots, x_{t-1}) = \text{softmax}(Wz_t + b) \quad (24)$$

donde $W \in \mathbb{R}^{|V| \times d}$ es la matriz de pesos de proyección al espacio del vocabulario de tamaño $|V|$, $b \in \mathbb{R}^{|V|}$ es un vector de sesgo, y *softmax* se define como:

$$\text{softmax}(u_i) = \frac{\exp(u_i)}{\sum_{j=1}^{|V|} \exp(u_j)} \quad (25)$$

De este modo, se obtiene una distribución de probabilidad sobre todos los *tokens* posibles del vocabulario, de la cual se elige uno según la estrategia de generación utilizada.

Este proceso se ejecuta de forma autorregresiva, generando un *token* por iteración. La salida generada se concatena a la secuencia de entrada y se vuelve a introducir en el modelo hasta alcanzar una longitud máxima predefinida o encontrar un *token* especial de parada (<eos> o similar).

Parámetros de ajuste de la salida

El modo en que se selecciona el siguiente *token* a partir de la distribución de probabilidad $P(x_t \mid x_{<t})$ afecta considerablemente a la calidad, diversidad y coherencia del texto generado. A continuación se describen las técnicas más comunes:

- **Temperature scaling** [13]: Consiste en modificar la distribución antes del muestreo aplicando un factor de temperatura $\tau > 0$ que ajusta la aleatoriedad. Se define una nueva distribución como:

$$P_\tau(x_t) = \frac{\exp\left(\frac{u_t}{\tau}\right)}{\sum_j \exp\left(\frac{u_j}{\tau}\right)} \quad (26)$$

donde $u_t = (Wz_t + b)_t$ es el *logit* asociado al *token* x_t . Valores bajos de τ ($\tau \rightarrow 0$) hacen que la distribución sea más "picuda", favoreciendo *tokens* con probabilidad más alta, mientras que valores altos ($\tau \rightarrow \infty$) la suavizan, aumentando la aleatoriedad.

- **Top-k sampling** [7]: Limita el espacio de muestreo a los k *tokens* más probables. Se define el conjunto $\mathcal{K} \subseteq V$ de los k *tokens* con mayor probabilidad según los *logits* originales, y se renormaliza la distribución:

$$P_k(x_t) = \begin{cases} \frac{P(x_t)}{\sum_{j \in \mathcal{K}} P(x_j)} & \text{si } x_t \in \mathcal{K} \\ 0 & \text{en otro caso} \end{cases} \quad (27)$$

Esto permite reducir la probabilidad de generar *tokens* inusuales o irrelevantes, a la vez que se mantiene cierta aleatoriedad dentro del conjunto de opciones más plausibles.

- **Top-p sampling (nucleus sampling)** [13]: Alternativamente, se define un subconjunto $\mathcal{P} \subseteq V$ que contiene los *tokens* más probables cuya probabilidad acumulada excede un umbral $p \in (0, 1]$. Formalmente:

$$\mathcal{P} = \left\{ x_i \in V \mid \sum_{j=1}^i P(x_j) \leq p \right\} \quad (28)$$

y se muestrea de forma similar a *top-k*, renormalizando dentro de $\mathcal{P}$. Esta técnica es más flexible, ya que el tamaño del subconjunto se adapta dinámicamente a la forma de la distribución.

La elección del mecanismo de muestreo influye en el equilibrio entre coherencia y diversidad. En la práctica, se suelen combinar varias técnicas (por ejemplo, *top-k* con temperatura) para ajustar el comportamiento generativo del modelo a tareas concretas.

4. Optimización en LLMs

El entrenamiento y despliegue de *Large Language Models* (LLMs) plantea importantes desafíos computacionales y metodológicos debido a su enorme tamaño y complejidad. En este apartado se analizan los principales retos asociados a su optimización, así como las técnicas más relevantes que permiten mejorar su eficiencia sin comprometer significativamente el rendimiento.

4.1. Principales retos y soluciones de optimización

Uno de los principales obstáculos en el desarrollo de LLMs es su **alto coste computacional y de memoria**. Estos modelos pueden tener hasta cientos de miles de millones de parámetros, lo que implica un consumo elevado de recursos tanto en el entrenamiento [25] como en la inferencia [16]. Para abordar esta cuestión, se han propuesto técnicas como la **cuantización**, que consiste en representar los pesos del modelo utilizando menos bits (por ejemplo, 8-bit en lugar de 32-bit) [6], y la **sparsity**, que busca eliminar conexiones innecesarias reduciendo la densidad de los tensores involucrados [21].

Otro problema relevante es la **dificultad para generalizar sin sobreajuste (*overfitting*)**. Dado el gran número de parámetros, los LLMs tienen una alta capacidad expresiva que puede llevar a memorizar el conjunto de entrenamiento si no se dispone de datos suficientemente diversos o si no se aplican técnicas de regularización adecuadas. Algunas

soluciones comunes incluyen el uso de *Dropout*, la incorporación de penalizaciones en la función de pérdida, como la regularización de tipo ℓ_2 , o el *data augmentation* textual para enriquecer el corpus [9, 34].

Además, en muchos contextos prácticos se requiere que los modelos sean capaces de adaptarse a **tareas específicas** sin necesidad de reentrenar toda la red neuronal, lo cual sería inviable desde el punto de vista computacional. Para ello, se emplean técnicas de *fine-tuning* eficiente, que modifican solo una pequeña fracción de los parámetros. Estas técnicas, conocidas como *Parameter-Efficient Fine-Tuning* (PEFT), serán estudiadas en profundidad en una sección posterior.

Otro desafío es la **obsolescencia del conocimiento** adquirido por el modelo, especialmente cuando el entorno cambia dinámicamente. Técnicas de *continual learning* u *online learning* permiten actualizar el conocimiento del modelo de forma incremental, evitando el *catastrophic forgetting* mediante estrategias específicas en la función de pérdida o en la arquitectura [23].

Por último, para ciertas aplicaciones es crítico reducir la **latencia en la inferencia**, especialmente en entornos con restricciones de tiempo real. Para ello, se emplean estrategias como la *knowledge distillation*, que entrena un modelo más pequeño (estudiante) a partir de la salida de uno más grande (profesor), o el *model pruning*, que elimina conexiones de baja importancia [12, 10].

En las siguientes secciones se profundiza en algunas de estas técnicas clave.

4.2. Técnicas clásicas de optimización

La **cuantización** consiste en aproximar los pesos del modelo, originalmente representados como números en coma flotante de 32 bits, por elementos de un conjunto discreto de menor precisión. Sea $w \in \mathbb{R}$ un peso, su versión cuantizada se obtiene mediante una función $Q : \mathbb{R} \rightarrow \mathcal{C}$, donde $\mathcal{C}$ es un subconjunto finito de $\mathbb{R}$. En la **cuantización uniforme**, se utiliza un paso fijo Δ tal que

$$Q(w) = \Delta \cdot \text{round} \left(\frac{w}{\Delta} \right), \quad (29)$$

mientras que en la **cuantización no uniforme** se diseñan niveles adaptados a la distribución de los pesos [6].

La técnica de **pruning** elimina conexiones consideradas no relevantes. Dado un conjunto de pesos $\{w_i\}$, se definen umbrales de magnitud τ y se anulan aquellos pesos tales

que $|w_i| < \tau$. El modelo resultante se vuelve esparso, lo que puede aprovecharse para reducir el coste computacional si el *hardware* soporta operaciones sobre tensores dispersos [10].

La **destilación de conocimiento** es un proceso en el que un modelo ligero (estudiante) aprende a imitar el comportamiento de un modelo complejo (profesor). Formalmente, se define una función de pérdida combinada [12]:

$$\mathcal{L}_{\text{KD}} = \alpha \cdot \mathcal{L}_{\text{hard}} + (1 - \alpha) \cdot \mathcal{L}_{\text{soft}}, \quad (30)$$

donde $\mathcal{L}_{\text{hard}}$ es la pérdida estándar con las etiquetas reales y $\mathcal{L}_{\text{soft}}$ mide la distancia entre las distribuciones de salida del profesor y el estudiante, normalmente mediante la entropía cruzada de distribuciones suavizadas:

$$\mathcal{L}_{\text{soft}} = \text{CE}(\sigma(z^{\text{prof}}/T), \sigma(z^{\text{est}}/T)), \quad (31)$$

donde σ es la función *softmax* y $T > 1$ es un parámetro de temperatura que suaviza las distribuciones.

4.3. Optimización del proceso de entrenamiento

Existen también técnicas orientadas a reducir el coste del entrenamiento de LLMs:

- **Warmup y schedulers de tasa de aprendizaje:** permiten evitar inestabilidades iniciales y mejorar la convergencia ajustando dinámicamente la tasa de aprendizaje, por ejemplo con esquemas como *cosine decay* [11].
- **Batch sizes adaptativos:** el tamaño del *batch* puede ajustarse a lo largo del entrenamiento para equilibrar estabilidad y eficiencia.
- **Gradient checkpointing:** técnica que permite reducir el uso de memoria durante el entrenamiento a cambio de un pequeño coste computacional adicional, recalculando ciertos valores intermedios en lugar de almacenarlos.
- **Mixed-precision training:** utiliza aritmética de menor precisión (como FP16) para acelerar las operaciones y reducir consumo de memoria, manteniendo la precisión en puntos críticos con FP32 [22].

4.4. *Fine-Tuning* de modelos

El ***fine-tuning*** es un proceso fundamental en el ciclo de vida de los LLMs, que permite adaptar un modelo preentrenado a una tarea o dominio específico. Tras una fase inicial de preentrenamiento masivo sobre grandes corpus de texto, estos modelos requieren ajustes adicionales para especializarse en tareas concretas. El *fine-tuning* cumple esta función, mejorando el rendimiento del modelo con un coste computacional y de datos menor que el entrenamiento desde cero.

El ***full fine-tuning*** es el enfoque clásico, en el cual se actualizan todos los parámetros del modelo base. Aunque permite una adaptación completa, tiene un coste muy elevado en términos computacionales y de almacenamiento.

El ***fine-tuning supervisado*** consiste en ajustar el modelo mediante un conjunto de datos etiquetado específico para una tarea concreta, como clasificación, traducción o resumen. Es ampliamente utilizado cuando se dispone de ejemplos claramente definidos y una métrica directa de evaluación.

A pesar de su efectividad, estos métodos tradicionales presentan problemas de escalabilidad. Por ello, han surgido técnicas más eficientes que permiten un ajuste parcial del modelo con una fracción de parámetros entrenables, agrupadas bajo el nombre de *Parameter-Efficient Fine-Tuning* (PEFT).

4.5. *Parameter-Efficient Fine-Tuning* (PEFT)

El paradigma de ***Parameter-Efficient Fine-Tuning*** (PEFT) surge como una respuesta directa a las limitaciones del *fine-tuning* clásico en modelos de gran tamaño. Su objetivo principal es adaptar un modelo preentrenado a nuevas tareas modificando únicamente una porción reducida de sus parámetros, minimizando así los costes de almacenamiento y computación. A continuación se presentan las técnicas más representativas dentro de este enfoque, cada una con mecanismos particulares para introducir capacidad de adaptación sin alterar la estructura completa del modelo base.

La técnica de ***adapters***, introducida por Houlsby et al. (2019) [14], consiste en insertar pequeñas capas entrenables entre las capas del modelo base. Estas capas aprenden transformaciones específicas de la tarea sin modificar los pesos originales. Gracias a este mecanismo, se puede compartir un único modelo base para múltiples tareas, simplemente activando los *adapters* correspondientes.

BitFit, propuesta por Zaken et al. (2022) [35], se basa en ajustar exclusivamente

los términos de sesgo (*bias*) del modelo, dejando todos los pesos principales congelados. A pesar de su simplicidad, ha mostrado resultados competitivos en tareas estándar de procesamiento de lenguaje natural.

La técnica conocida como *prefix tuning*, presentada por Li y Liang (2021) [19], introduce vectores entrenables al comienzo de la entrada en cada capa del transformador. Estos vectores, denominados *prefixes*, actúan como una guía contextual que orienta la atención del modelo hacia ciertas interpretaciones deseadas.

Finalmente, la técnica **LoRA** (*Low-Rank Adaptation of Large Language Models*), propuesta por Hu et al. (2021) [15], inserta matrices de bajo rango entrenables en algunas proyecciones lineales del modelo, lo que permite ajustar su comportamiento con una cantidad muy reducida de parámetros adicionales.

Estas técnicas han demostrado que es posible adaptar modelos de lenguaje de manera eficaz y eficiente, reduciendo sustancialmente los requisitos computacionales sin perder capacidad expresiva. Entre ellas, LoRA que se analizará con mayor detalle en el siguiente apartado, ha destacado por su simplicidad, rendimiento y escalabilidad, siendo aplicable incluso en modelos con más de 20 mil millones de parámetros. Esta superioridad relativa se refleja en estudios comparativos como el de Lialin et al. (2023) [18], donde se recogen las características clave de las principales técnicas PEFT, incluyendo el porcentaje de parámetros entrenables, la magnitud de los cambios introducidos y la viabilidad en modelos de diferentes tamaños (véase la Figura 5).

Method	%Trainable parameters	%Changed parameters	Evaluated on		
			<1B	<20B	>20B
Adapters	0.1 - 6	0.1 - 6	✓	✓	✓
AdaMix	0.1 - 0.2	0.1 - 0.2	✓	✗	✗
SparseAdapter	2.0	2.0	✓	✗	✗
BitFit	0.05 - 0.1	0.05 - 0.1	✓	✓	✓
DiffPruning	200	0.5	✓	✗	✗
Fish-Mask	0.01 - 0.5	0.01 - 0.5	✓	✓	✗
Prompt Tuning	0.1	0.1	✓	✓	✓
Prefix-Tuning	0.1 - 4.0	0.1 - 4.0	✓	✓	✓
IPT	56.0	56.0	✓	✗	✗
MAM Adapter	0.5	0.5	✓	✗	✗
Parallel Adapter	0.5	0.5	✓	✗	✗
Intrinsic SAID	0.001 - 0.1	100	✓	✓	✗
LoRa	0.01 - 0.5	~ 30	✓	✓	✓
DoRA	0.01 - 0.5	~ 30	✗	✓	✗
UniPELT	1.0	1.0	✓	✗	✗
Compacter	0.05-0.07	~ 0.1	✓	✓	✗
PHM Adapter	0.2	~ 1.0	✓	✗	✗
KronA	0.07	~ 30.0	✓	✗	✗
KronA _{res} ^B	0.07	~ 1.0	✓	✗	✗
(IA) ³	0.02	0.02	✗	✓	✗
Ladder Side-Tuning	7.5	7.5	✓	✓	✗
FAR	6.6-26.4	6.6-26.4	✓	✗	✗
S4-model	0.5	> 0.5	✓	✓	✗

Figura 5: Tabla 2 de [18] con comparación del porcentaje de parámetros entrenables en distintos métodos PEFT.

5. Técnica *Low-Rank Adaptation* (LoRA)

Como se ha argumentado en el apartado anterior, una de las principales barreras para la implementación práctica de los LLMs es su enorme coste computacional, tanto en entrenamiento como en inferencia. Este obstáculo se acentúa al intentar adaptar modelos preentrenados a nuevas tareas específicas mediante *fine-tuning* completo, lo cual implica reoptimizar millones o incluso miles de millones de parámetros. En este contexto, han surgido técnicas de *Parameter-Efficient Fine-Tuning* (PEFT), entre las cuales destaca LoRA, por su simplicidad conceptual, solidez matemática y eficacia empírica (véase la Figura 6 para una comparación visual entre ambos enfoques).

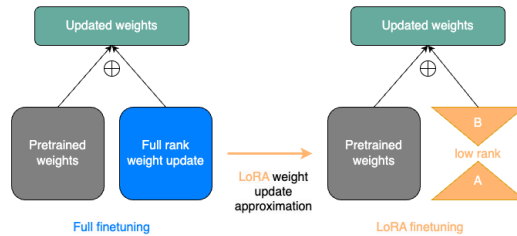


Figura 6: Esquema comparativo entre el *fine-tuning* completo y la técnica LoRA.

La técnica *Low-Rank Adaptation* (LoRA) ofrece una solución elegante y eficiente que permite adaptar modelos grandes a nuevas tareas mediante la inserción de matrices adicionales de bajo rango, evitando la necesidad de actualizar los parámetros originales del modelo.

5.1. Formulación matemática y eficiencia de LoRA

Supongamos que tenemos una matriz de pesos $W_0 \in \mathbb{R}^{d \times k}$ ya preentrenada. El *fine-tuning* convencional implicaría modificar W_0 directamente, optimizando todos sus $d \cdot k$ parámetros. Sin embargo, cuando d y k son del orden de miles (como ocurre en capas de atención de modelos grandes), el coste computacional se vuelve prohibitivo.

La técnica LoRA, propuesta por Hu et al. (2021) [15], aborda este problema al mantener fija la matriz W_0 y añadir una perturbación entrenable $\Delta W \in \mathbb{R}^{d \times k}$, tal que:

$$W = W_0 + \Delta W. \quad (32)$$

La innovación central consiste en imponer que ΔW tenga **rango bajo**, representándola como el producto de dos matrices más pequeñas:

$$\Delta W = BA, \quad \text{con } B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k). \quad (33)$$

Esta parametrización tiene una doble ventaja. Por un lado, permite capturar información direccional relevante en un subespacio de dimensión reducida, limitando la capacidad del modelo de forma controlada y, al mismo tiempo, manteniendo su capacidad de adaptación. Por otro lado, dado que la matriz original W_0 permanece congelada durante todo el proceso de ajuste, el modelo base se conserva intacto. LoRA actúa añadiendo una actualización de bajo rango $\Delta W = BA$, que puede interpretarse como una parametrización diferenciada de la LLM específica para la tarea de destino.

De este modo, es posible reutilizar el mismo modelo base preentrenado con múltiples matrices ΔW , cada una adaptada a una tarea concreta. Esto habilita un paradigma modular en el que una única LLM sirve como base común y las capacidades específicas se inyectan a través de las matrices entrenables BA , lo que resulta especialmente útil en entornos con recursos limitados o donde se desea compartir el conocimiento común entre tareas sin incurrir en costes de almacenamiento o entrenamiento redundantes.

Además, LoRA introduce un factor de escalado α/r sobre la actualización para estabilizar el entrenamiento:

$$W = W_0 + \frac{\alpha}{r} BA. \quad (34)$$

La elección de una descomposición de la forma BA se justifica algebraicamente mediante el teorema de factorización de rango. Si $\Delta W \in \mathbb{R}^{d \times k}$ tiene rango r , entonces puede expresarse como suma de r productos exteriores:

$$\Delta W = \sum_{i=1}^r \mathbf{b}_i \mathbf{a}_i^\top, \quad \mathbf{b}_i \in \mathbb{R}^d, \quad \mathbf{a}_i \in \mathbb{R}^k. \quad (35)$$

Esto equivale a la representación matricial $\Delta W = BA$ con B y A de dimensiones $d \times r$ y $r \times k$, donde las columnas de B y las filas de A agrupan los vectores $\mathbf{b}_i$ y $\mathbf{a}_i^\top$, respectivamente.

Desde un punto de vista geométrico, esta restricción equivale a desplazar W_0 dentro de una subvariedad lineal de dimensión $r(d+k)$, en lugar de permitir cualquier desplazamiento dentro del espacio completo $\mathbb{R}^{d \times k}$. Esta reducción de grados de libertad actúa

como una forma de regularización implícita: al reducir la dimensión efectiva del espacio de hipótesis, se disminuye la complejidad del modelo y su propensión al sobreajuste, mejorando la generalización especialmente en tareas con pocos ejemplos.

Además, el hecho de que W_0 permanezca congelado permite preservar el conocimiento adquirido en el preentrenamiento, mitigando el efecto conocido como *catastrophic forgetting*.

Ejemplo de LoRA con matrices pequeñas

Para visualizar el funcionamiento de *LoRA* (*Low-Rank Adaptation*) en un entorno simple y controlado, consideramos una capa lineal representada por una matriz de pesos $W \in \mathbb{R}^{3 \times 3}$, sobre la cual aplicamos una adaptación de bajo rango sin modificar directamente los pesos originales. El objetivo es comparar tres casos:

1. Modelo sin LoRA ($y = Wx$).
2. Modelo con LoRA sin escalado ($y = (W + \Delta W)x$ con $\Delta W = BA$).
3. Modelo con LoRA con escalado ($y = (W + \frac{\alpha}{r}BA)x$).

Partimos de una entrada fija $x \in \mathbb{R}^3$:

$$x = \begin{bmatrix} 1,0 \\ 2,0 \\ -1,0 \end{bmatrix} \quad (36)$$

Y de la siguiente matriz de pesos congelada:

$$W = \begin{bmatrix} 0,2 & -0,1 & 0,4 \\ 0,0 & 0,3 & -0,2 \\ -0,5 & 0,1 & 0,6 \end{bmatrix} \quad (37)$$

La salida sin LoRA se obtiene como:

$$y = Wx = \begin{bmatrix} -0,4 \\ 0,8 \\ -0,9 \end{bmatrix} \quad (38)$$

Para la actualización LoRA, utilizamos un rango $r = 1$ y definimos las matrices:

$$A = \begin{bmatrix} 0,1 & 0,2 & -0,3 \end{bmatrix}, \quad B = \begin{bmatrix} 0,5 \\ -0,4 \\ 1,0 \end{bmatrix} \quad (39)$$

El producto $\Delta W = BA$ da lugar a la siguiente corrección:

$$\Delta W = \begin{bmatrix} 0,05 & 0,10 & -0,15 \\ -0,04 & -0,08 & 0,12 \\ 0,10 & 0,20 & -0,30 \end{bmatrix} \quad (40)$$

Sumando esta corrección a la matriz original se obtiene la matriz efectiva con LoRA sin escalar:

$$W_{\text{LoRA}} = W + \Delta W = \begin{bmatrix} 0,25 & 0,00 & 0,25 \\ -0,04 & 0,22 & -0,08 \\ -0,40 & 0,30 & 0,30 \end{bmatrix} \quad (41)$$

Y la salida correspondiente es:

$$y_{\text{LoRA}} = W_{\text{LoRA}} \cdot x = \begin{bmatrix} 0,0 \\ 0,48 \\ -0,1 \end{bmatrix} \quad (42)$$

A continuación, aplicamos un factor de escalado $\alpha = 32$, de modo que:

$$\Delta W_{\text{esc}} = \frac{32}{1} \cdot \Delta W = \begin{bmatrix} 1,6 & 3,2 & -4,8 \\ -1,28 & -2,56 & 3,84 \\ 3,2 & 6,4 & -9,6 \end{bmatrix} \quad (43)$$

La matriz de pesos adaptada con escalado es:

$$W_{\text{esc}} = W + \Delta W_{\text{esc}} = \begin{bmatrix} 1,8 & 3,1 & -4,4 \\ -1,28 & -2,26 & 3,64 \\ 2,7 & 6,5 & -9,0 \end{bmatrix} \quad (44)$$

Y su salida correspondiente:

$$y_{\text{esc}} = W_{\text{esc}} \cdot x = \begin{bmatrix} 12,4 \\ -9,44 \\ 24,7 \end{bmatrix} \quad (45)$$

La evolución de la salida y en los tres casos puede resumirse como:

$$\begin{array}{ccc} \begin{bmatrix} -0,4 \\ 0,8 \\ -0,9 \end{bmatrix} & \longrightarrow & \begin{bmatrix} 0,0 \\ 0,48 \\ -0,1 \end{bmatrix} & \longrightarrow & \begin{bmatrix} 12,4 \\ -9,44 \\ 24,7 \end{bmatrix} \\ \text{Sin LoRA} & & \text{LoRA sin escalar} & & \text{LoRA escalado} \end{array} \quad (46)$$

Este experimento permite observar tres comportamientos claramente diferenciados:

- La salida sin LoRA representa el comportamiento original del modelo.
- La versión con LoRA sin escalar aplica una modificación leve al modelo, alterando la salida sin desviarse excesivamente. Esta opción es útil cuando se requiere una adaptación sutil a una nueva tarea relacionada.
- La versión con LoRA escalada genera una salida drásticamente distinta. Este escenario puede ser útil cuando se busca una adaptación más fuerte del modelo base, pero también puede implicar riesgos si el escalado es excesivo.

En resumen, el uso del factor de escalado α permite controlar la magnitud de la contribución de la corrección de LoRA. Este ejemplo, aunque simplificado, refleja de forma intuitiva cómo se modifica el comportamiento de una capa lineal utilizando una actualización de bajo rango sin modificar directamente los pesos preentrenados.

A partir de esta construcción simple, podemos extrapolar el beneficio de LoRA a matrices de mayor dimensión como las que se encuentran en modelos de lenguaje actuales. En estos casos, la principal ventaja práctica radica en la drástica reducción del número de parámetros entrenables, lo que habilita un *fine-tuning* mucho más eficiente tanto en coste de cómputo como en consumo de memoria.

Supongamos que trabajamos con una matriz $W_0 \in \mathbb{R}^{d \times k}$, y aplicamos una adaptación de rango r . Entonces:

$$\text{Fine-tuning completo: } d \cdot k \text{ parámetros} \longrightarrow \text{LoRA: } r(d + k) \text{ parámetros.}$$

Por ejemplo, si $d = k = 2048$ y $r = 8$, se tiene:

$$\text{Fine-tuning completo: } 2048^2 = 4\,194\,304 \text{ parámetros,}$$

$$\text{LoRA: } 8(2048 + 2048) = 32\,768 \text{ parámetros.}$$

Este ahorro superior al 99 % permite realizar *fine-tuning* con una fracción del coste de memoria y cálculo, posibilitando el entrenamiento de adaptaciones específicas en dispositivos personales, servidores compartidos o incluso en paralelo sobre múltiples tareas. Además, durante la inferencia, las matrices BA pueden fusionarse con W_0 para no penalizar el rendimiento, lo que hace que LoRA sea también eficiente en tiempo de ejecución.

5.2. Hiperparámetros fundamentales en LoRA

El comportamiento y la eficacia de la técnica LoRA dependen en gran medida de la configuración de ciertos hiperparámetros clave. Estos controlan tanto la capacidad del submodelo entrenable como la intensidad de la adaptación sobre el modelo base. Elegir adecuadamente estos valores es esencial para lograr una buena generalización sin incurrir en sobreajuste ni en costes computacionales innecesarios.

A continuación se describen los hiperparámetros más relevantes de la configuración estándar de LoRA (implementación basada en `LoraConfig`), detallando su función, impacto en el rendimiento y recomendaciones de uso.

El primer parámetro esencial es el **rango r** , que determina la dimensión del espacio de adaptación de bajo rango. Cuanto mayor es el valor de r , mayor es la capacidad de LoRA para modificar el comportamiento del modelo base, pero también aumenta el número de parámetros entrenables. En concreto, este número es $r(d + k)$ para cada capa adaptada. Valores pequeños de r (por ejemplo, 4 u 8) son adecuados para tareas sencillas o cuando se desea un entrenamiento muy eficiente. Valores mayores (16, 32...) permiten más flexibilidad en tareas complejas, pero incrementan el coste computacional.

Otro hiperparámetro fundamental es el **factor de escalado α** , que regula la magnitud de la perturbación introducida por LoRA. Su efecto se refleja en el término de actualización $\frac{\alpha}{r}BA$, donde un valor elevado de α puede desestabilizar el entrenamiento al introducir una modificación demasiado agresiva sobre W_0 . En la práctica, se utilizan valores entre 16 y 64, aunque pueden ajustarse en función del tamaño del modelo y la dificultad de la tarea.

El ***dropout* `lora_dropout`** permite introducir regularización sobre la actualización BA durante el entrenamiento. Es útil cuando se dispone de pocos datos o cuando se quiere evitar el sobreajuste en tareas sensibles. Aunque el valor por defecto es 0.0 (sin regularización), valores entre 0.05 y 0.2 son comunes en entornos con alta varianza.

El hiperparámetro ***bias*** determina si se entrenan o no los términos de sesgo de las capas afectadas por LoRA. Las opciones más frecuentes son:

- “none”: no se entrena ningún sesgo adicional.
- “lora_only”: se entrena el sesgo solo en las capas donde se aplica LoRA.
- “all”: se entrenan todos los sesgos del modelo.

En entornos de baja capacidad computacional o cuando se busca la máxima eficiencia, se recomienda usar la opción por defecto “none”.

El hiperparámetro `target_modules` indica los módulos concretos del modelo sobre los que se aplica LoRA. En modelos *Transformer*, lo habitual es adaptar las matrices de atención (por ejemplo, “q” y “v”), aunque también puede aplicarse a capas *feed-forward* (como “dense”). Seleccionar adecuadamente estos módulos permite un control fino del grado de intervención de LoRA sobre el modelo original.

Por último, el hiperparámetro `fan_in_fan_out` ajusta automáticamente la orientación del producto BA en arquitecturas donde las convenciones de pesos requieren una inversión, como en algunas capas convolucionales. En modelos tipo *Transformer*, típicamente se deja como `False`, ya que no se necesita esta inversión.

Para una visión general de los principales hiperparámetros disponibles en la implementación de LoRA, junto con una breve descripción de su función, véase el Anexo A.

5.3. Aplicación en arquitecturas *Transformer*

En los modelos tipo *Transformer*, LoRA se aplica típicamente en las matrices de proyección del mecanismo de atención, que son aquellas que generan las consultas, claves y valores a partir de la entrada X :

$$Q = XW_q, \quad K = XW_k, \quad V = XW_v. \quad (47)$$

Con LoRA, las matrices W_q , W_k , etc., se sustituyen por versiones modificadas:

$$W_q = W_q^0 + \frac{\alpha}{r} B_q A_q, \quad W_k = W_k^0 + \frac{\alpha}{r} B_k A_k, \quad \text{etc.} \quad (48)$$

En la implementación habitual, B y A se inicializan aleatoriamente, y B puede ser fija (por ejemplo, una matriz identidad extendida) para reducir aún más los parámetros entrenables. Además, LoRA puede aplicarse selectivamente solo a ciertas capas o subcomponentes del modelo, como las capas de atención o de *feed-forward*, dependiendo del presupuesto computacional y la sensibilidad de la tarea.

5.4. Casos de uso y eficacia empírica

Numerosos trabajos recientes han evidenciado la eficacia de LoRA en tareas diversas. Por ejemplo:

- **Alpaca** (Taori et al., 2023): adaptación del modelo LLaMA mediante LoRA para tareas de instrucción con resultados competitivos y coste computacional ínfimo [30].
- **QLoRA** (Dettmers et al., 2023): combina LoRA con cuantización en 4 bits, permitiendo entrenar modelos de hasta 65B parámetros en una sola GPU [5].
- **DreamBooth LoRA** (Gal et al., 2022): adaptación eficiente de modelos de difusión para generación de imágenes [8].

Estos casos no solo demuestran la viabilidad técnica de LoRA, sino que también destacan que el rendimiento de los modelos adaptados con esta técnica puede igualar —e incluso superar— al *fine-tuning* completo cuando se entrena adecuadamente y con buenos datos.

6. Diseño y desarrollo del caso práctico

El presente estudio práctico tiene como objetivo comparar el rendimiento de distintos modelos de aprendizaje automático sobre un conjunto de datos reales del sector bancario. Concretamente, se evaluarán modelos clásicos de *Machine Learning* ampliamente utilizados —regresión logística, árboles de decisión, *random forest*, *XGBoost* y redes neuronales—, un modelo de lenguaje de gran tamaño (*Large Language Model*, LLM) aplicado a datos tabulares mediante transformación textual, y una versión afinada de dicho LLM utilizando la técnica de adaptación de bajo rango conocida como LoRA (*Low-Rank Adaptation*).

Este enfoque experimental permite contrastar la eficiencia y capacidad predictiva de modelos tradicionales frente a técnicas modernas basadas en redes neuronales profundas. Además, la introducción de LoRA proporciona una perspectiva adicional sobre la posibilidad de adaptar LLMs a tareas específicas con recursos computacionales reducidos, alineándose directamente con los objetivos generales del trabajo y con los fundamentos teóricos tratados en secciones anteriores. Todo el código empleado para el desarrollo del caso práctico, así como los experimentos realizados, se encuentra disponible públicamente en el repositorio de GitHub: <https://github.com/Daniseralz/TFG-LLM-LoRA>.

6.1. Descripción del conjunto de datos

El conjunto de datos empleado en este estudio proviene de una campaña de *marketing* directo llevada a cabo por una entidad bancaria portuguesa entre mayo de 2008 y noviembre de 2010, cuyo objetivo era ofrecer depósitos a plazo fijo mediante llamadas telefónicas. El *dataset* original contiene un total de 41.188 registros, cada uno correspondiente a un cliente contactado por el banco, y 21 variables que describen sus características: edad, estudios, estado civil, situación financiera, historial de contacto y respuesta a la campaña, entre otras.

Este conjunto de datos está disponible públicamente en la plataforma Kaggle bajo el título *Bank Marketing Dataset* (2018), accesible en: <https://www.kaggle.com/datasets/henriqueyamahata/bank-marketing>. Consultado en 2025.

Dado que el *dataset* original sólo incluye el mes y no el año de contacto, se ha construido una variable temporal adicional denominada **Date**, con el fin de ordenar cronológicamente los registros. Para ello, se ha llevado a cabo un análisis externo que ha consistido en comparar el mes declarado con la evolución histórica del tipo de interés **Euribor** durante el periodo 2008–2010, estimando así el año más probable asociado a cada observación. Esto ha permitido construir una serie temporal coherente con la cronología real de la campaña, facilitando tanto el análisis exploratorio como la división temporal del conjunto de datos.

Debido a que los modelos de lenguaje empleados (**Flan-T5** base y su versión adaptada con LoRA) operan sobre texto en inglés, se ha decidido mantener los nombres de las variables y los valores categóricos en su idioma original. Esta elección también permite aprovechar mejor el conocimiento aprendido durante el preentrenamiento del modelo sobre corpus anglófonos.

El análisis se ha centrado en un subconjunto del *dataset* correspondiente al periodo en el que el Euribor se encontraba en niveles bajos. Este subconjunto consta de 16.349 registros, lo cual reduce el tamaño del conjunto de datos, facilitando así los experimentos computacionales. Además, presenta una proporción más equilibrada entre las clases **yes** y **no** de la variable objetivo, lo cual resulta especialmente adecuado para tareas de clasificación binaria.

6.2. Evaluación de modelos y estrategias de validación

La evaluación de modelos en aprendizaje automático es una etapa fundamental que permite estimar la capacidad de generalización de un modelo más allá de los datos con los

que ha sido entrenado. Elegir correctamente las métricas y la metodología de validación es esencial para evitar conclusiones engañosas y para tomar decisiones adecuadas respecto a la idoneidad del modelo en una tarea concreta. Además, permite comparar distintos algoritmos bajo criterios objetivos y cuantificables.

En esta sección se describen los principales métodos de validación utilizados para estimar el rendimiento de un modelo, junto con diversas métricas de evaluación, destacando su utilidad práctica y sus limitaciones.

División del dataset y validación

■ Separación en conjuntos de entrenamiento, validación y prueba

Una práctica habitual consiste en dividir el conjunto de datos disponible en tres subconjuntos:

- **Conjunto de entrenamiento** (*training set*): utilizado para ajustar los parámetros del modelo.
- **Conjunto de validación** (*validation set*): empleado para ajustar los hiperparámetros y evitar sobreajuste.
- **Conjunto de prueba** (*test set*): reservado para, una vez entrenado y validado el modelo, evaluar su rendimiento final sobre datos no vistos.

Una proporción comúnmente adoptada es 60 %-20 %-20 %, aunque en tareas con componente temporal esta división puede adaptarse para respetar la cronología. En la Tabla 2 se muestra la distribución concreta utilizada en este trabajo para cada partición.

	<i>Train</i>	<i>Validation</i>	<i>Test</i>
<i>Split 1</i>	Mar. 09 - Sep. 09 9467 (68.4 %)	Oct. 09 - Dic. 09 1812 (13.1 %)	Mar. 10 - May. 10 2553 (18.5 %)
<i>Split 2</i>	Jun. 09 - Dic. 09 3763 (45.9 %)	Mar. 10 - May. 10 2553 (31.2 %)	Jun. 10 - Ago. 10 1877 (22.9 %)
<i>Split 3</i>	Sep. 09 - May. 10 4572 (64.5 %)	Jun. 10 - Ago. 10 1877 (26.5 %)	Sep. 10 - Nov. 10 640 (9.0 %)

Tabla 2: Distribución de datos en cada partición (meses, número de instancias y porcentaje).

(*underfitting*). Al separar claramente los periodos de entrenamiento, validación y prueba en bloques temporales no solapados, se evita que el modelo acceda indirectamente a información futura durante el entrenamiento y se garantiza una evaluación más realista y fiable de su capacidad de generalización.

El **sobreajuste** ocurre cuando el modelo se ajusta excesivamente a los datos de entrenamiento, incluyendo ruido o valores atípicos, lo que afecta negativamente a su rendimiento en datos nuevos. Por el contrario, el **subajuste** se produce cuando el modelo es demasiado simple y no logra capturar las relaciones esenciales presentes en los datos, mostrando bajo rendimiento tanto en el conjunto de entrenamiento como en el de prueba.

Estos conceptos se ilustran visualmente en la Figura 8, que muestra el comportamiento típico de un modelo infraajustado, bien ajustado y sobreajustado en un problema de predicción.

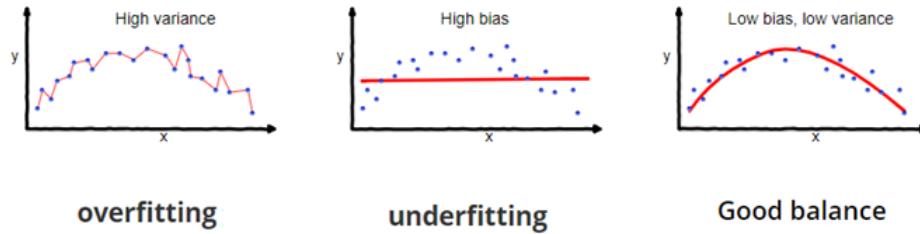


Figura 8: Representación gráfica de underfitting, good fitting y overfitting.

Para mitigar estos problemas, se emplean técnicas de regularización que introducen restricciones o modificaciones en el proceso de entrenamiento con el objetivo de mejorar la generalización. Algunas de las más utilizadas son:

- **Regularización L2 (*Ridge*):** penaliza la norma cuadrada de los pesos del modelo, favoreciendo soluciones con parámetros de menor magnitud:

$$Loss_{\text{reg}} = Loss + \lambda \|\mathbf{w}\|_2^2 \quad (49)$$

- **Regularización L1 (*Lasso*):** penaliza la suma de valores absolutos de los pesos, promoviendo soluciones dispersas donde algunos parámetros pueden anularse:

$$Loss_{\text{reg}} = Loss + \lambda \|\mathbf{w}\|_1 \quad (50)$$

- **Dropout:** técnica habitual en redes neuronales que consiste en desactivar aleatoriamente un porcentaje de neuronas durante cada iteración de entrenamiento, forzando al modelo a no depender de activaciones individuales.
- **Early stopping:** consiste en monitorizar el rendimiento sobre el conjunto de validación y detener el entrenamiento tan pronto como la pérdida comienza a aumentar, lo que indica que el modelo empieza a sobreajustar los datos de entrenamiento.

Métricas de evaluación

Una vez entrenado y validado un modelo, es necesario cuantificar su rendimiento mediante métricas adecuadas. En este trabajo se utilizan varias métricas estándar ampliamente empleadas en tareas de clasificación binaria, que permiten valorar diferentes aspectos del comportamiento del modelo. Estas métricas se calculan a partir de la matriz de confusión, que organiza las predicciones del modelo en función de su coincidencia con las etiquetas reales.

Dado que el objetivo del presente estudio es predecir si un cliente contratará un depósito bancario, se define como clase positiva el evento **yes** (el cliente contrata el producto) y como clase negativa el evento **no** (el cliente no lo contrata). A partir de esta clasificación, se establecen los siguientes conceptos:

- **TP (True Positives):** casos en los que el modelo predice **yes** y el cliente efectivamente contrata el depósito.
- **TN (True Negatives):** casos en los que el modelo predice **no** y el cliente realmente no contrata el depósito.
- **FP (False Positives):** predicciones de **yes** cuando el cliente en realidad no contrata el depósito.
- **FN (False Negatives):** predicciones de **no** cuando el cliente sí contrataba el depósito.

A continuación se describen las métricas seleccionadas para evaluar el rendimiento de los modelos utilizados:

Exactitud (*Accuracy*)

El *accuracy* mide la proporción de predicciones correctas sobre el total de predicciones realizadas:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (51)$$

Es una métrica intuitiva y global, pero puede resultar engañosa en contextos con clases desbalanceadas, ya que un modelo que siempre predice la clase mayoritaria puede obtener una alta exactitud sin ser útil.

Precisión (*Precision*)

La precisión evalúa qué proporción de las predicciones positivas realizadas por el modelo son correctas:

$$Precision = \frac{TP}{TP + FP} \quad (52)$$

Es especialmente relevante cuando el coste de los falsos positivos es elevado. En el contexto bancario, una baja precisión implicaría ofrecer el depósito a clientes que no lo contratarán, incurriendo en esfuerzos de *marketing* poco rentables.

Exhaustividad (*Recall*)

El *recall*, o sensibilidad, indica qué proporción de los clientes que realmente contratarían un depósito han sido correctamente identificados por el modelo:

$$Recall = \frac{TP}{TP + FN} \quad (53)$$

Es útil cuando es importante no omitir casos positivos. En este caso, un alto *recall* implica identificar correctamente a los clientes con alta probabilidad de contratación, maximizando el alcance de las campañas exitosas.

F1-Score

El *F1-Score* combina precisión y *recall* en una única métrica mediante su media armónica, equilibrando ambos factores:

$$F1-Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (54)$$

Resulta especialmente útil en situaciones con clases desbalanceadas, como ocurre en este caso, donde los clientes que contratan el depósito (**yes**) son minoría.

Exactitud balanceada (*Balanced Accuracy*)

La *balanced accuracy* corrige la limitación de la exactitud clásica al considerar por igual ambas clases. Se define como la media entre la sensibilidad (*recall*) para la clase positiva y la especificidad (*recall* para la clase negativa):

$$\text{Balanced Accuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right) \quad (55)$$

Permite evaluar el rendimiento global del modelo incluso cuando las clases están desbalanceadas, proporcionando una visión más justa que la exactitud tradicional.

Finalmente, aunque durante la optimización bayesiana de los modelos de *machine learning* se ha utilizado el *F1-Score* como métrica a maximizar, la comparación final de los resultados se ha realizado utilizando *precision* y *recall*. Estas métricas han sido seleccionadas por su mayor claridad interpretativa y por su especial relevancia en contextos donde es crucial controlar tanto los falsos positivos como los falsos negativos, lo que las hace más adecuadas para un análisis práctico.

6.3. Modelos de *Machine Learning*

Como punto de partida para la comparación, se han entrenado y evaluado diversos modelos de aprendizaje automático clásicos, ampliamente utilizados en tareas de clasificación binaria. Todos los experimentos se han llevado a cabo en entornos de programación reproducibles utilizando **Jupyter Notebook** y el lenguaje **Python**, lo que ha permitido documentar y controlar todas las fases del proceso.

Los modelos considerados en este estudio son los siguientes:

- **Regresión logística:** modelo estadístico lineal que estima la probabilidad de pertenencia a una clase mediante una función sigmoide. Es interpretable y sirve como línea base en clasificación binaria.
- **Árboles de decisión:** modelo no paramétrico que construye reglas de decisión jerárquicas segmentando el espacio de características. Permite capturar relaciones no lineales y es fácilmente interpretable.
- **Random Forest:** técnica de ensamblado basada en *bagging*, que combina múltiples árboles de decisión entrenados sobre subconjuntos aleatorios del conjunto de datos. Mejora la robustez y reduce la varianza.
- **XGBoost:** algoritmo de *boosting* que construye árboles secuencialmente, optimizando una función de pérdida diferenciable en cada iteración. Es conocido por su alto rendimiento en tareas con datos tabulares y estructuras complejas.

- **Red neuronal multicapa (MLP):** arquitectura de red *feedforward* compuesta por una o más capas ocultas densas, donde cada neurona aplica una transformación no lineal a una combinación lineal de sus entradas. Este tipo de modelo permite aproximar funciones complejas y capturar relaciones no lineales entre las variables.

Todos los modelos han sido evaluados utilizando la misma estrategia de validación cruzada temporal basada en ventana deslizante, con *splits* definidos por bloques mensuales (véase Figura 7). Para cada modelo y cada *split*, se ha seguido un protocolo sistemático compuesto por los siguientes pasos:

1. Entrenamiento inicial del modelo base sobre el conjunto de entrenamiento.
2. Evaluación preliminar en el conjunto de validación para establecer un punto de referencia.
3. Optimización de hiperparámetros mediante búsqueda bayesiana, implementada mediante la librería `Optuna`, tomando como métrica objetivo el *F1-Score* sobre el conjunto de validación.
4. Reentrenamiento del modelo con los hiperparámetros óptimos utilizando únicamente los datos de entrenamiento.
5. Evaluación del modelo ajustado tanto en entrenamiento como en validación, con el objetivo de detectar posibles signos de sobreajuste o subajuste.
6. Reentrenamiento final sobre la combinación de entrenamiento y validación, aprovechando toda la información disponible antes de la evaluación final.
7. Evaluación definitiva del modelo sobre el conjunto de prueba, utilizando todas las métricas descritas previamente.

Este procedimiento se repitió en cada uno de los tres *splits* definidos, garantizando una evaluación cronológicamente coherente y libre de fugas de información. A partir de los resultados obtenidos en cada *split*, se han calculado las métricas medias por modelo, que serán utilizadas en la comparación final con los modelos LLM, tanto en su configuración estándar como ajustados mediante la técnica LoRA.

6.4. Modelos LLM y adaptación con LoRA

Además de los modelos clásicos de aprendizaje automático, en este trabajo se ha evaluado el rendimiento de un modelo de lenguaje de gran tamaño (LLM) aplicado a datos tabulares mediante transformación textual. Asimismo, se ha explorado el impacto de aplicar técnicas de ajuste eficiente como LoRA (*Low-Rank Adaptation*) sobre dicho modelo, con el fin de mejorar su rendimiento sin incurrir en grandes costes computacionales.

El modelo seleccionado en ambos casos ha sido **Flan-T5 base**, una variante del modelo T5 preentrenada por *Google* con técnicas de *instruction tuning*. Se trata de un modelo *encoder-decoder* de tipo *seq2seq* originalmente diseñado para tareas como traducción, resumen o respuesta a preguntas, pero que puede aplicarse a otros dominios mediante entradas adecuadas.

Adaptación textual del *dataset* tabular

Dado que el modelo espera entradas textuales, se transformó cada instancia del conjunto de datos en una estructura de tipo **instruction-output**, en la que el campo **instruction** contiene una descripción en inglés de las variables del cliente, y el campo **output** corresponde a la clase objetivo (**yes** o **no**). Un ejemplo representativo es el siguiente:

```
{“instruction”: “age: 18, job: student, marital status: single, education:
high.school, credit in default: no, housing loan: yes, personal loan:
yes, contact: cellular, day week: tue, duration: 103, campaign: 1,
days without contact: 999, number previous contacts: 0, previous campaign
result: nonexistent, employment variation rate: -1.8, consumer price
index: 92843.0, consumer confidence index: -50.0, euribor: 1687, employees:
5099.1, Date: March 2009, Will they take out a deposit?”,
“output”: “no”}
```

Durante la inferencia, tanto en el modelo base como en el adaptado con LoRA, el modelo únicamente recibe como entrada el campo **“instruction”**. La respuesta generada por el modelo se compara con el valor del campo **“output”** para calcular las métricas de evaluación. Es decir, durante la predicción, el modelo no tiene acceso a la respuesta correcta.

Sin embargo, en el caso del modelo ajustado con LoRA, el campo **“output”** sí se utiliza durante la fase de entrenamiento para que el modelo aprenda a generar la salida esperada a partir de la entrada textual. Esta diferencia es clave: el modelo base (**Flan-T5**) actúa

como un generador preentrenado sin ver nunca las etiquetas, mientras que el modelo con LoRA se entrena explícitamente para predecirlas a partir de ejemplos etiquetados.

Entorno de ejecución

Todos los experimentos, tanto con el modelo base (Flan-T5) como con su versión adaptada mediante LoRA, se realizaron en la plataforma Google Colab, utilizando exclusivamente CPU. El entorno proporciona una CPU virtual del tipo Intel(R) Xeon(R) CPU @ 2.20GHz, con 2 núcleos y aproximadamente 13.6 GB de memoria RAM. Dado que no se dispuso de GPU, se adoptó una configuración ligera y optimizada, con tamaños de lote reducidos, número limitado de épocas y desactivación del guardado de *checkpoints* intermedios.

Las herramientas comunes utilizadas incluyen:

- **transformers** y **datasets** (*Hugging Face*): para la carga, tokenización y preprocesamiento de los datos y modelos.
- **scikit-learn**: para el cálculo de las métricas de evaluación (*accuracy*, *precision*, *recall*, *F1-Score*, etc.).

En el caso de LoRA, se emplearon además:

- **peft**: para insertar módulos LoRA en el modelo base.
- **Trainer** y **TrainingArguments**: para controlar el ciclo de entrenamiento y evaluación de cada split.

6.4.1. Modelo LLM sin ajuste

En primer lugar, se evaluó el modelo Flan-T5 base en modo inferencia, sin aplicar ningún tipo de *fine-tuning*, con el objetivo de analizar su rendimiento únicamente con el conocimiento adquirido durante el preentrenamiento. Esta evaluación se realizó exclusivamente sobre CPU, lo que supuso tiempos de ejecución considerables: entre 1 y 2 horas⁴ por *split*, siendo especialmente costosa sobre el conjunto *train* debido al mayor número de instancias.

Para cada uno de los tres *splits* temporales definidos, se utilizaron por separado los conjuntos de entrenamiento, validación y prueba, y se realizaron predicciones directamente

⁴Los tiempos detallados de entrenamiento y evaluación por split se recogen en la Tabla 8 del Anexo B.

sobre cada uno. El modelo generaba una respuesta textual, de la que se extraía el primer *token* (**yes** o **no**) como predicción de clase.

A modo ilustrativo, el proceso de inferencia en el modelo **Flan-T5** base sigue las siguientes etapas:

- **Entrada textual:** El modelo recibe una instrucción en formato textual estructurado, por ejemplo:

```
{instruction: age: 18, job: student, marital status: single, education: high.school, credit in default: no, housing loan: yes, personal loan: yes, contact: cellular, day week: tue, duration: 103, campaign: 1, days without contact: 999, number previous contacts: 0, previous campaign result: nonexistent, employment variation rate: -1.8, consumer price index: 92843.0, consumer confidence index: -50.0, euribor: 1687, employees: 5099.1, Date: March 2009, Will they take out a deposit?}
```
- **Tokenización:** Esta cadena se convierte en una secuencia de identificadores numéricos (*input IDs*) mediante el tokenizador del modelo, basado en el algoritmo *Unigram Language Model*. Por ejemplo: [204, 83, 14, ..., 1].
- **Embeddings:** Cada *token* se transforma en un vector denso mediante una capa de *embedding*, que codifica tanto el contenido semántico como la posición dentro de la secuencia.
- **Procesamiento *encoder-decoder*:** Los vectores de entrada se procesan a través del *encoder* y *decoder* del modelo T5, generando distribuciones de probabilidad sobre posibles secuencias de salida.
- **Decodificación:** El modelo genera como respuesta una secuencia textual, comenzando típicamente por “yes” o “no”. Para propósitos de clasificación binaria, se toma únicamente el primer *token* como predicción final.

Este procedimiento se aplica sobre cada instancia del conjunto de datos, permitiendo evaluar el comportamiento del modelo en modo inferencia pura, sin ajuste específico a la tarea y actúa como línea base frente a su versión adaptada con LoRA.

6.4.2. Modelo LLM ajustado con LoRA

Para mejorar el rendimiento del modelo anterior, se aplicó la técnica LoRA, que permite ajustar parámetros de forma eficiente insertando matrices de bajo rango en las capas

de atención, congelando el resto de parámetros del modelo original. El entrenamiento con LoRA se realizó en dos etapas por *split*: primero sobre el conjunto *train*, y posteriormente sobre la unión *train+validation*, antes de evaluarse en *test*. Esta fase completa —entrenamiento más evaluación— requirió entre 4 y 5 horas⁵ por división temporal, ejecutándose íntegramente sobre CPU.

Se utilizó la función `get_peft_model` sobre `Flan-T5 base` con la siguiente configuración:

- `r`: 16
- `lora_alpha`: 32
- `lora_dropout`: 0.0
- `bias`: "none"
- `target_modules`: {"q", "k", "v", "o", "wi", "wo"}

El protocolo seguido por cada *split* fue el siguiente:

1. Entrenamiento del modelo con LoRA sobre el conjunto de entrenamiento.
2. Evaluación en el conjunto de validación.
3. Reentrenamiento con los conjuntos de entrenamiento y validación combinados.
4. Evaluación final sobre el conjunto de prueba.

Selección reducida y balanceada para el entrenamiento con LoRA

Para reducir los tiempos de entrenamiento y hacer viable el ajuste en CPU, se emplearon subconjuntos reducidos y balanceados de los datos disponibles. En cada *split*, se seleccionaron únicamente 500 individuos del conjunto de entrenamiento, compuestos por 250 ejemplos de la clase “yes” y 250 de la clase “no”. Esta decisión permitió acelerar significativamente el proceso sin comprometer la representatividad de las clases.

De forma análoga, durante el reentrenamiento con el conjunto de entrenamiento y validación combinados, se tomaron 500 ejemplos de cada uno, también balanceados por clase, dando lugar a un conjunto total de 1000 ejemplos (500 yes y 500 no). Este enfoque

⁵Los tiempos detallados de entrenamiento y evaluación por *split* se recogen en la Tabla 8 del Anexo B.

mantuvo una distribución equilibrada de etiquetas en ambas fases, lo que favorece una señal de entrenamiento robusta y reduce el riesgo de sobreajuste, especialmente importante en contextos con recursos computacionales limitados.

Durante el entrenamiento, se monitorizó la evolución de la función de pérdida, observándose un descenso progresivo a medida que avanzaban los pasos de optimización. Este comportamiento refleja una mejora continua del ajuste del modelo a los datos de entrenamiento. En la Figura 9 se muestra un ejemplo de dicha evolución en el conjunto `train1`, donde se aprecia cómo la *training loss* disminuye de forma significativa durante las primeras fases del entrenamiento hasta estabilizarse en torno a un valor bajo.



Figura 9: Evolución de la función de pérdida durante el entrenamiento del modelo LoRA sobre el conjunto `train1`.

La salida generada por el modelo se interpretó del mismo modo que en el caso base. Las métricas utilizadas fueron idénticas, lo que permite una comparación directa entre el rendimiento del modelo preentrenado y el adaptado con LoRA.

6.5. Resultados y comparativa entre modelos

6.5.1. Comparación en entrenamiento, validación y test de modelos clásicos

Comparación en entrenamiento

Para analizar el rendimiento de los modelos clásicos de *Machine Learning* se evaluó primero su comportamiento sobre el conjunto de entrenamiento sin ajuste de hiperparámetros, y posteriormente con los hiperparámetros optimizados mediante búsqueda bayesiana (*Optuna*). Las métricas consideradas fueron: *Accuracy*, *Precision*, *Recall*, *F1-Score*

y *Balanced Accuracy*.

La siguiente tabla muestra el rendimiento medio de cada modelo en el conjunto *train*, tanto sin ajustar como tras el ajuste de hiperparámetros:

Modelo	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1-Score</i>	<i>Balanced Acc.</i>
Regresión Logística	0.8104 → 0.7831	0.6822 → 0.6166	0.4663 → 0.5192	0.5506 → 0.5354	0.6942 → 0.6936
Árbol de Decisión	1.0000 → 0.8189	1.0000 → 0.6889	1.0000 → 0.5288	1.0000 → 0.5822	1.0000 → 0.7192
<i>Random Forest</i>	1.0000 → 0.9427	1.0000 → 0.9419	1.0000 → 0.8387	1.0000 → 0.8871	1.0000 → 0.9085
<i>XGBoost</i>	0.9758 → 0.8610	0.9658 → 0.7694	0.9252 → 0.6401	0.9447 → 0.6963	0.9576 → 0.7852
Red Neuronal	0.8226 → 0.9723	0.7136 → 0.9310	0.4846 → 0.9681	0.5708 → 0.9483	0.7071 → 0.9709

Tabla 3: Evolución de métricas en entrenamiento tras ajuste de hiperparámetros.

Además, en la Figura 10 se representa de forma visual la evolución de las métricas para cada modelo con y sin ajuste, lo que permite apreciar mejor las diferencias obtenidas tras el *tuning*.

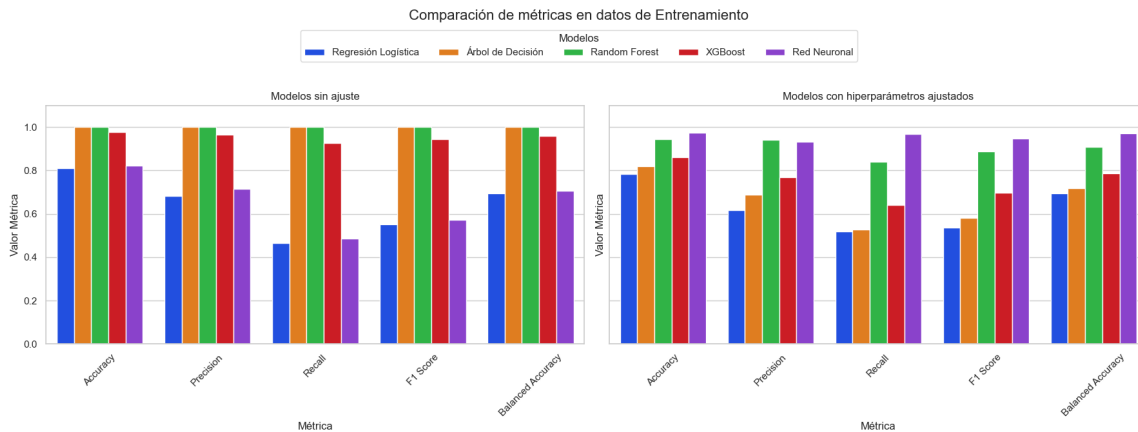


Figura 10: Comparación de métricas sobre datos de entrenamiento (con y sin ajuste).

Como se observa, el modelo *XGBoost* ya presentaba un rendimiento excelente sin ajuste, mientras que en otros casos como la *Red Neuronal*, el ajuste mejoró significativamente todas las métricas. También se aprecia que los modelos que sobreajustan (como el Árbol de Decisión y *Random Forest* con todas las métricas perfectas) se estabilizan al introducir el ajuste bayesiano, mejorando su capacidad de generalización, lo cual se evaluará en los siguientes apartados con los conjuntos de validación y test.

Comparación en validación

Una vez ajustados los hiperparámetros mediante búsqueda bayesiana, se evaluó el comportamiento de los modelos sobre el conjunto de *validation*. En la Tabla 4 se muestra la evolución de cada métrica para los distintos modelos, comparando directamente el valor obtenido sin ajuste frente al obtenido tras el ajuste.

Modelo	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1-Score</i>	<i>Balanced Acc.</i>
Regresión Logística	0.8132 → 0.8150	0.6234 → 0.6138	0.5728 → 0.5830	0.5912 → 0.5978	0.7293 → 0.7338
Árbol de Decisión	0.6768 → 0.7878	0.4148 → 0.5732	0.5342 → 0.5520	0.4478 → 0.5518	0.6265 → 0.7076
<i>Random Forest</i>	0.7411 → 0.7661	0.5410 → 0.5648	0.6293 → 0.6406	0.5480 → 0.5703	0.7036 → 0.7236
<i>XGBoost</i>	0.7497 → 0.7769	0.5268 → 0.5624	0.6741 → 0.6913	0.5698 → 0.6002	0.7234 → 0.7488
Red Neuronal	0.8148 → 0.5783	0.6270 → 0.3249	0.5670 → 0.6592	0.5895 → 0.4335	0.7263 → 0.6039

Tabla 4: Evolución de métricas en validación tras ajuste de hiperparámetros.

En la Figura 11 se presentan las gráficas correspondientes a las métricas de los modelos, separando entre los resultados sin ajuste (izquierda) y los resultados con hiperparámetros ajustados (derecha), facilitando la observación de las tendencias generales de mejora tras el ajuste.

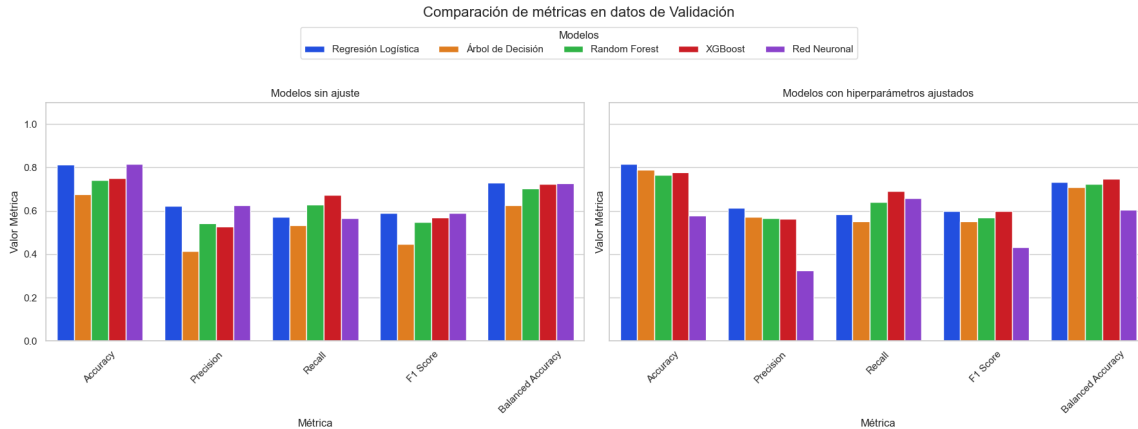


Figura 11: Comparación de métricas sobre datos de validación (con y sin ajuste).

A diferencia del comportamiento observado en el conjunto de entrenamiento, los modelos no presentan signos de sobreajuste en validación. Se observa una mejora generalizada tras el ajuste de hiperparámetros en la mayoría de modelos, especialmente en el Árbol de Decisión y *XGBoost*, que logran incrementos notables en todas las métricas clave, como *Precision*, *F1-Score* y *Balanced Accuracy*.

Por otro lado, la *Red Neuronal* muestra una ligera disminución de rendimiento tras el ajuste, lo que podría indicar una configuración subóptima de hiperparámetros o una alta sensibilidad a los mismos. En cambio, modelos como *Regresión Logística* y *Random Forest* mantienen una evolución estable, consolidando su rendimiento en validación sin grandes fluctuaciones.

Estas observaciones refuerzan la importancia de validar el comportamiento real de los modelos en datos no vistos, y muestran cómo el ajuste de hiperparámetros, aunque beneficioso en la mayoría de los casos, no garantiza siempre mejoras en todos los modelos ni métricas.

Comparación en test

La última evaluación se realizó sobre el conjunto test, con el objetivo de valorar la capacidad de generalización real de los modelos tras el entrenamiento y ajuste. En la Tabla 5 se recoge la evolución de las métricas principales, comparando directamente los resultados antes y después del ajuste de hiperparámetros.

Modelo	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1-Score</i>	<i>Balanced Acc.</i>
Regresión Logística	0.8209 → 0.7522	0.7468 → 0.6676	0.5682 → 0.5913	0.6219 → 0.5875	0.7202 → 0.6628
Árbol de Decisión	0.6905 → 0.8179	0.4878 → 0.6684	0.6037 → 0.7629	0.5285 → 0.7036	0.6591 → 0.7755
<i>Random Forest</i>	0.7829 → 0.8004	0.6164 → 0.6378	0.6677 → 0.6956	0.6261 → 0.6505	0.7368 → 0.7549
<i>XGBoost</i>	0.7684 → 0.8250	0.5886 → 0.6973	0.7036 → 0.6737	0.6295 → 0.6830	0.7456 → 0.7769
Red Neuronal	0.8290 → 0.7670	0.7313 → 0.5541	0.6072 → 0.6447	0.6573 → 0.5958	0.7472 → 0.7153

Tabla 5: Evolución de métricas en test tras ajuste de hiperparámetros.

La Figura 12 muestra de forma gráfica las métricas obtenidas por cada modelo antes (izquierda) y después (derecha) del ajuste.

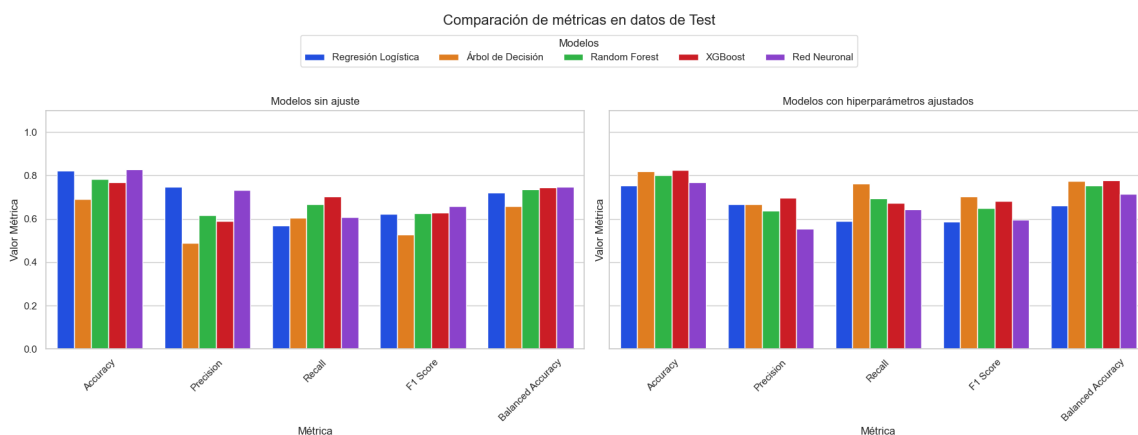


Figura 12: Comparación de métricas sobre datos de test (con y sin ajuste).

En este caso, se observa que el ajuste de hiperparámetros tiene un efecto muy distinto según el modelo. Mientras que el rendimiento de la *Red Neuronal* empeora notablemente en todas las métricas, modelos como el *Árbol de Decisión*, *XGBoost* y *Random Forest* muestran mejoras claras, especialmente en *Recall*, *F1-Score* y *Balanced Accuracy*.

El *Árbol de Decisión*, que había tenido un comportamiento más modesto en validación, logra tras el ajuste un rendimiento muy competitivo en test. En cambio, la *Regresión Logística* pierde parte de su eficacia tras el ajuste, lo que indica que su configuración inicial ya era bastante adecuada.

Estos resultados subrayan que el ajuste de hiperparámetros debe evaluarse cuidadosamente sobre el conjunto de validación, ya que no siempre conduce a una mejora uniforme. También refuerzan la importancia de comparar múltiples modelos y métricas para obtener conclusiones sólidas sobre el comportamiento de los algoritmos.

6.5.2. Comparación entre LLM y LLM ajustado con LoRA

Además de los modelos clásicos de aprendizaje automático, se evaluaron dos variantes de modelos de lenguaje grande: el modelo **Flan-T5 base** sin ajustar, y su versión ajustada mediante la técnica LoRA. En ambos casos se entrenaron y evaluaron sobre los mismos splits temporales, con el objetivo de comparar su rendimiento de forma consistente.

En la Figura 13 se muestra la evolución de las métricas estándar (*Accuracy*, *Precision*, *Recall*, *F1-Score* y *Balanced Accuracy*) en las tres fases: entrenamiento, validación y test.

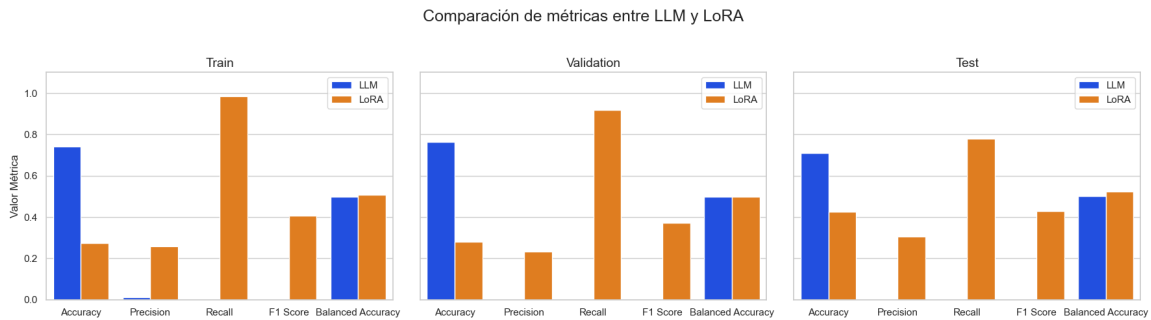


Figura 13: Comparación de métricas por fase entre LLM y LoRA.

Se observa que el modelo **Flan-T5 base** obtiene buenos valores de *Accuracy*, pero presenta valores nulos o muy bajos en *Recall*, *F1-Score* y *Precision*, lo que indica un comportamiento fuertemente sesgado hacia la clase mayoritaria.

Por el contrario, al aplicar LoRA, se produce un aumento significativo en métricas como *Recall* y *F1-Score* en todas las fases, especialmente en test, lo que indica una mejora notable en la capacidad del modelo para detectar correctamente la clase minoritaria.

Esta mejora se produce a costa de una ligera reducción en *Accuracy*, pero con un efecto beneficioso sobre el *Balanced Accuracy*, lo que sugiere un mejor equilibrio en la predicción entre clases.

En conjunto, los resultados demuestran que el uso de LoRA permite adaptar de forma efectiva un modelo de lenguaje generalista a una tarea de clasificación binaria específica, mejorando sustancialmente la calidad de las predicciones sin necesidad de ajustar todos los parámetros del modelo.

6.5.3. Comparación final

Como síntesis final de la evaluación de modelos, se presenta una comparación directa de las métricas *Precision* y *Recall* sobre el conjunto de test para todos los modelos considerados en este trabajo: los modelos clásicos, el modelo **Flan-T5 base** (LLM) y su versión ajustada mediante LoRA.

En el caso de los modelos clásicos, se ha seleccionado para cada uno la versión —ajustada o no— que obtuvo la mayor puntuación de *F1-Score* durante la validación. Por tanto, se seleccionan las **versiones ajustadas** de Regresión Logística, Árbol de Decisión, *Random Forest* y *XGBoost*, y la **versión base** de Red Neuronal, al ser esta la que ofrece mejor rendimiento en términos de *F1-Score*. Estos cinco modelos serán los utilizados para la evaluación final en el conjunto de test.

Por su parte, los modelos LLM y LoRA solo disponen de una única evaluación en test, por lo que se han incluido directamente sus valores finales sin necesidad de selección previa.

La Figura 14 muestra la comparación visual de *Precision* y *Recall* entre todos los modelos.

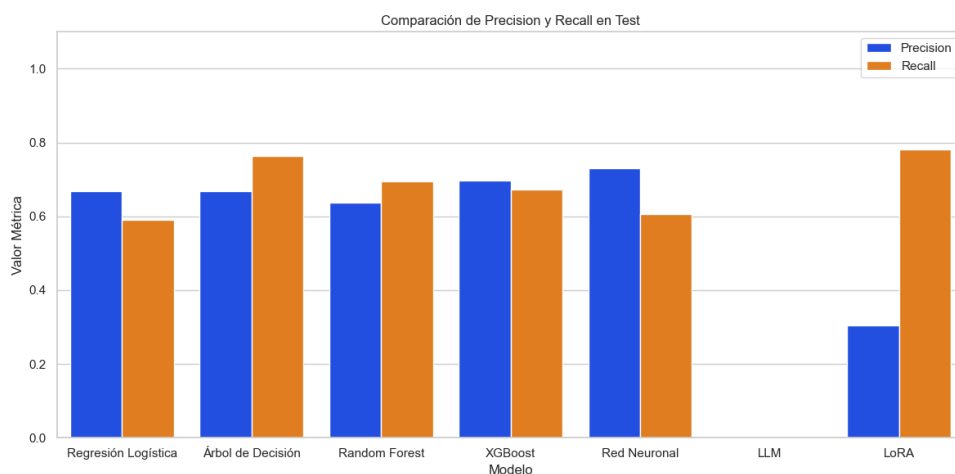


Figura 14: Comparación de *Precision* y *Recall* en test entre modelos clásicos, LLM y LoRA.

La gráfica permite observar de forma clara el comportamiento de cada modelo en términos de equilibrio entre estas dos métricas. Se puede apreciar que el modelo LoRA presenta un *Recall* significativamente alto (0.78), lo que indica una gran capacidad de detección de la clase minoritaria, aunque con una precisión más modesta. Por el contrario, el modelo LLM no logra identificar correctamente instancias positivas, lo que se refleja en valores cercanos a cero.

Entre los modelos clásicos, destaca el Árbol de Decisión, que logra un alto valor tanto en *Precision* como en *Recall*, seguido por *XGBoost* y Red Neuronal. Esta comparación permite constatar que el ajuste con LoRA es especialmente efectivo para mejorar la capacidad de detección del modelo base, aunque en términos de equilibrio general, algunos modelos clásicos siguen presentando un rendimiento competitivo.

7. Conclusiones

El presente trabajo ha abordado la optimización de modelos de lenguaje de gran tamaño (LLMs) mediante técnicas de ajuste eficiente, con el objetivo de adaptarlos a tareas de clasificación específicas sin incurrir en elevados costes computacionales. La investigación ha combinado una revisión teórica detallada con un estudio experimental riguroso, estructurado en torno a tres pilares fundamentales: los fundamentos del aprendizaje profundo, el análisis de arquitecturas *encoder-decoder*, y la técnica de adaptación LoRA.

Desde el punto de vista teórico, se ha descrito cómo los LLMs aprenden representaciones lingüísticas generales a través del preentrenamiento auto-supervisado. Se han comparado los paradigmas de modelado de lenguaje causal y enmascarado, con especial atención al enfoque de *span corruption* empleado en T5. Además, se ha analizado el proceso de optimización mediante retropropagación, detallando la formulación del gradiente, la función de coste y el papel de la regla de la cadena en redes profundas, tal y como se expone en Goodfellow et al. (2016) [9].

La técnica LoRA ha sido el eje central del apartado de optimización. Se ha presentado como una estrategia de *fine-tuning* eficiente que permite insertar matrices de bajo rango en ciertas capas del modelo sin modificar los parámetros originales. Este enfoque resulta especialmente útil en entornos con recursos limitados, como ha sido el caso en este trabajo, donde los entrenamientos se han realizado íntegramente en CPU.

En la parte práctica, se ha construido un pipeline completo en **Python** para comparar modelos clásicos de aprendizaje automático con un LLM (**Flan-T5 base**) y su versión afinada con LoRA. El estudio se ha realizado sobre un conjunto de datos reales del sector bancario, aplicando validación cruzada temporal, balanceo de clases y evaluación con

múltiples métricas (*accuracy*, *precision*, *recall*, *F1-Score*, *balanced accuracy*).

Los resultados obtenidos permiten extraer las siguientes conclusiones:

- **Los modelos clásicos**, especialmente tras el ajuste de hiperparámetros, han alcanzado un rendimiento sólido y equilibrado en test, con valores de *accuracy*, *precision* y *recall* superiores al 70 % en la mayoría de los casos. Entre ellos destacan: árbol de decisión, *XGBoost* y redes neuronales.
- El **modelo LLM base**, usado en modo inferencia sin ajuste, ha mostrado un comportamiento muy limitado. Tal como se observa en la Figura 13, su precisión es prácticamente nula, ya que apenas predice la clase positiva. Esto se traduce también en una pérdida de *recall* y de *F1-Score*, lo que pone de manifiesto que, pese a su potencia generalista, un LLM preentrenado requiere adaptación para tareas concretas con distribución distinta.
- Al aplicar **LoRA** sobre el LLM, se produce una transformación significativa en su comportamiento. En todas las fases (entrenamiento, validación y test), el modelo afinado mejora considerablemente el *recall*, alcanzando valores cercanos al 80 % en test. Esta mejora viene acompañada de un aumento en el *F1-Score* y en la *balanced accuracy*, lo cual es especialmente relevante en contextos con clases desbalanceadas.
- La técnica LoRA ha demostrado ser eficaz para ajustar modelos grandes sin reentrenar todos sus parámetros, permitiendo mejorar el rendimiento con un consumo muy reducido de recursos. Además, ha aportado estabilidad y consistencia en las métricas clave, compensando la baja sensibilidad del modelo base.

En resumen, este trabajo ha evidenciado que las técnicas de ajuste eficiente como LoRA permiten aprovechar el poder representacional de los LLMs sin necesidad de acceso a *hardware* especializado. En comparación con los modelos tradicionales, los LLMs ajustados con LoRA no solo ofrecen una solución competitiva, sino que también son capaces de adaptarse mejor a tareas donde la detección de la clase positiva es prioritaria, como en la predicción de respuestas a campañas bancarias. Además, LoRA permite personalizar un mismo modelo de lenguaje para múltiples tareas específicas sin modificar sus parámetros originales, lo que habilita la posibilidad de mantener un único LLM preentrenado y múltiples adaptadores ligeros para distintas aplicaciones, reduciendo drásticamente los costes de almacenamiento y despliegue en entornos productivos.

8. Trabajos futuros

Como líneas futuras de trabajo se plantean:

- **Explorar otras técnicas *PEFT*.** Aunque en este trabajo se ha utilizado exclusivamente LoRA, existen otras estrategias de ajuste eficiente como *Adapters*, *Prefix Tuning* o *BitFit*. Sería interesante comparar estas técnicas en condiciones homogéneas (mismo modelo base, tarea y conjunto de datos), evaluando no solo el rendimiento predictivo, sino también aspectos como el tiempo de entrenamiento, el número de parámetros ajustados o la facilidad de implementación. Por ejemplo, se podría implementar *Prefix Tuning* sobre el modelo `Flan-T5` y analizar si ofrece ventajas frente a LoRA en tareas de clasificación binaria con datos financieros.
- **Aplicar el análisis a modelos LLM más grandes o especializados.** En este trabajo se ha utilizado un modelo base generalista (`Flan-T5 Base`), pero cabría extender el estudio a modelos de mayor tamaño (como `Flan-T5 Large`) o a modelos específicamente preentrenados en lenguaje financiero (por ejemplo, `FinBERT` o `BloombergGPT`). Esto permitiría evaluar si la especialización previa del modelo mejora significativamente el rendimiento tras un ajuste fino con *PEFT* en tareas del dominio bancario.

Cabe señalar que en este trabajo no se han utilizado modelos especializados en lenguaje financiero, como `FinBERT` o `BloombergGPT`, debido a limitaciones tanto de disponibilidad como de recursos. En el caso de `FinBERT`, si bien está disponible públicamente, su arquitectura está basada en *BERT* y no es directamente compatible con algunas de las técnicas *PEFT* implementadas para modelos *encoder-decoder* como `Flan-T5`. Por otro lado, `BloombergGPT` no se encuentra disponible para uso abierto, lo que impide su evaluación directa. Además, el uso de modelos de gran tamaño habría requerido una infraestructura computacional superior a la disponible en este proyecto. Por todo ello, se optó por trabajar con modelos accesibles y suficientemente versátiles como `Flan-T5 base`, priorizando la reproducibilidad y la viabilidad computacional.

- **Incluir criterios económicos y de eficiencia energética.** Más allá de las métricas de rendimiento clásico (*accuracy*, *F1-Score*, etc.), sería relevante cuantificar el coste computacional de cada enfoque. Por ejemplo, se podría medir el consumo energético (con herramientas como `codecarbon` o usando datos del proveedor *cloud*) y estimar el coste monetario del ajuste frente al entrenamiento completo. Esto permitiría construir una métrica coste-beneficio global que incorpore tanto eficacia

como eficiencia. En esta línea, iniciativas recientes como **DeepSeek** han destacado por entrenar grandes modelos desde cero con un enfoque optimizado en términos de eficiencia energética y coste computacional, demostrando que es posible escalar modelos de gran tamaño con un consumo mucho más contenido. Esta perspectiva puede complementar el uso de técnicas como LoRA, que también buscan maximizar el rendimiento sin necesidad de reentrenar todo el modelo.

- **Investigar enfoques híbridos entre modelos clásicos y LLMs.** Dado que los modelos clásicos suelen ser más interpretables y rápidos, mientras que los LLMs destacan por su capacidad de contextualización y generalización, una línea prometedora sería desarrollar arquitecturas híbridas. Por ejemplo, se podría usar un LLM ajustado con LoRA para generar nuevas variables explicativas a partir del texto de clientes o productos financieros, y luego alimentar esas variables a un modelo clásico como *XGBoost* o *Random Forest*, evaluando si se mejora la capacidad predictiva y la interpretabilidad del sistema combinado.

Referencias

- [1] Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.
- [2] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33. Disponible en: <https://arxiv.org/pdf/2005.14165>
- [3] Chung, H. W., Hou, L., Longpre, S., Zoph, B., Tay, Y., Fedus, W., Li, E., Wang, X., Lazard, A., et al. (2022). Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*.
- [4] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. En *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (Vol. 1, pp. 4171–4186).
- [5] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. *arXiv preprint arXiv:2305.14314*.
- [6] Dettmers, T., et al. (2022). LLM.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*.
- [7] Fan, A., Lewis, M., & Dauphin, Y. (2018). Hierarchical neural story generation. En *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics* (pp. 889–898).
- [8] Gal, R., Alaluf, Y., Atzmon, M., Patashnik, O., Bermano, A. H., Chechik, G., & Cohen-Or, D. (2022). DreamBooth: Personalizing text-to-image models. *arXiv preprint arXiv:2208.01618*.
- [9] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- [10] Han, S., Pool, J., Tran, J., & Dally, W. J. (2015). Learning both weights and connections for efficient neural network. En *Advances in Neural Information Processing Systems*.
- [11] He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385*.

-
- [12] Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.
 - [13] Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The curious case of neural text degeneration. En *International Conference on Learning Representations (ICLR)*.
 - [14] Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., Attariyan, M., & Gelly, S. (2019). Parameter-efficient transfer learning for NLP. En *Proceedings of the 36th International Conference on Machine Learning (ICML)*.
 - [15] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, L., Chen, W., & Rajaraman, A. (2021). LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.
 - [16] Kim, S., et al. (2023). Efficient inference for large language models. En *Proceedings of the MLSys Conference*.
 - [17] Kudo, T. (2018). Subword regularization: Improving neural network translation models with multiple subword candidates. *arXiv preprint arXiv:1804.10959*.
 - [18] Lialin, V., Deshpande, V., & Rumshisky, A. (2023). Scaling down to scale up: A guide to parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.15647*.
 - [19] Li, X. L., & Liang, P. (2021). Prefix-tuning: Optimizing continuous prompts for generation. En *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*.
 - [20] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., ... & Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.
 - [21] Liu, S., Li, Z., Zhou, S., Yu, F., Wu, Y., & Sun, Y. (2023). A survey of model compression and acceleration for pretrained language models. *ACM Computing Surveys*.
 - [22] Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., ... & Tang, H. (2018). Mixed precision training. En *International Conference on Learning Representations (ICLR)*.

- [23] Parisi, G. I., Kemker, R., Part, J. L., Kanan, C., & Wermter, S. (2019). Continual lifelong learning with neural networks: A review. *Neural Networks*, 113, 54–71.
- [24] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 1–67.
- [25] Rajbhandari, S., Rasley, J., Ruwase, O., & He, Y. (2021). ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning. *arXiv preprint arXiv:2104.07857*.
- [26] Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training. OpenAI. Disponible en: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf
- [27] Schuster, M., & Nakajima, K. (2012). Japanese and Korean voice search. En *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*.
- [28] Sennrich, R., Haddow, B., & Birch, A. (2015). Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*.
- [29] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., ... & Scao, T. L. (2023). LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- [30] Taori, R., Zhang, X., Dubois, Y., Guestrin, C., Hashimoto, T. B., & Liang, P. (2023). Alpaca: A strong, replicable instruction-following model. Stanford Center for Research on Foundation Models.
- [31] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.
- [32] Voita, E., Talbot, D., Moiseev, F., Sennrich, R., & Titov, I. (2019). Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. En *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 5797–5808).

-
- [33] Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. R. (2018). GLUE: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*.
 - [34] Wei, J., & Zou, K. (2019). EDA: Easy data augmentation techniques for text classification. En *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
 - [35] Zaken, E., Ravfogel, S., & Goldberg, Y. (2022). BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language models. En *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*.

A. Hiperparámetros ajustables en LoRa y en el entrenamiento

En esta sección se recogen los principales hiperparámetros que pueden modificarse tanto en la clase `LoraConfig` (que define la arquitectura y comportamiento de la técnica LoRA usada para ajuste eficiente de modelos de lenguaje) como en la clase `TrainingArguments` (que regula el proceso de entrenamiento con la librería `transformers`)

A.1. Hiperparámetros de `LoraConfig`

Hiperparámetro	Tipo	Descripción
<code>r</code>	entero	Rango de las matrices LoRA de bajo rango. Define la dimensión intermedia de la descomposición lineal. A mayor <code>r</code> , mayor capacidad de ajuste.
<code>lora_alpha</code>	entero	Factor de escalado aplicado al producto AB en LoRA. Influye en la magnitud de las actualizaciones.
<code>target_modules</code>	lista de cadenas	Lista de submódulos del modelo original a los que se insertarán capas LoRA, como <code>q</code> , <code>k</code> , <code>v</code> , etc.
<code>lora_dropout</code>	real	Probabilidad de <i>dropout</i> aplicada a la salida LoRA como forma de regularización.
<code>bias</code>	cadena	Define si los sesgos (<i>bias</i>) se entrenan o no: “none”, “all”, “lora_only”.
<code>task_type</code>	enumerado	Tipo de tarea del modelo: <code>SEQ_2_SEQ_LM</code> , <code>CAUSAL_LM</code> , <code>TOKEN_CLS</code> , etc.
<code>modules_to_save</code>	lista de cadenas	Opcionalmente, módulos adicionales del modelo base que también se desean guardar o ajustar.
<code>inference_mode</code>	booleano	Si está activo, el modelo desactiva todo lo relacionado con entrenamiento y opera en modo inferencia.
<code>init_lora_weights</code>	booleano	Si es <code>True</code> , inicializa los pesos LoRA con pequeñas perturbaciones.
<code>fan_in_fan_out</code>	booleano	Ajusta el orden de multiplicación en las matrices para compatibilidad con arquitecturas específicas.

A.2. Hiperparámetros de TrainingArguments

Hiperparámetro	Tipo	Descripción
<code>output_dir</code>	cadena	Carpeta donde se guardarán los resultados del entrenamiento, como pesos del modelo y <i>logs</i> .
<code>per_device_train_batch_size</code>	entero	Número de ejemplos que se procesan por paso de entrenamiento en cada dispositivo (CPU o GPU). Afecta directamente al uso de memoria.
<code>num_train_epochs</code>	entero	Número de veces que se recorre completamente el conjunto de entrenamiento.
<code>learning_rate</code>	real	Tasa inicial de aprendizaje usada por el optimizador para actualizar los parámetros entrenables.
<code>gradient_accumulation_steps</code>	entero	Número de pasos durante los cuales se acumulan gradientes antes de actualizar los pesos, útil cuando el <i>batch size</i> es pequeño.
<code>weight_decay</code>	real	Coefficiente de penalización L2 para evitar sobreajuste (regularización).
<code>adam_beta1 / beta2</code>	real	Coefficientes de amortiguación usados en el optimizador Adam.
<code>adam_epsilon</code>	real	Pequeño valor para evitar divisiones por cero en Adam.
<code>logging_steps</code>	entero	Frecuencia (en pasos) con la que se imprimen las métricas de entrenamiento.
<code>save_strategy</code>	cadena	Estrategia de guardado: “no”, “steps” o “epoch”. Controla cuándo se guardan <i>checkpoints</i> .
<code>evaluation_strategy</code>	cadena	Determina cuándo se realiza evaluación automática: “no”, “steps” o “epoch”.
<code>report_to</code>	cadena o lista	Define el <i>backend</i> para <i>logging</i> y seguimiento: “none”, “wandb”, “tensorboard”, etc.
<code>seed</code>	entero	Semilla aleatoria para reproducibilidad del entrenamiento.

B. Tiempos de entrenamiento y evaluación LLM y LoRA

Modelo	Fase	<i>Split 1</i>	<i>Split 2</i>	<i>Split 3</i>
LLM	Evaluar <i>train</i>	1:18:17	0:54:17	0:37:28
	Evaluar <i>val</i>	0:14:56	0:36:34	0:15:11
	Evaluar <i>test</i>	0:20:54	0:26:45	0:05:10
LoRA	Entrenar <i>train</i>	0:49:22	1:10:33	0:51:04
	Evaluar <i>train</i>	1:40:35	0:56:34	0:53:45
	Evaluar <i>val</i>	0:19:10	0:38:05	0:22:05
	Entrenar <i>train+val</i>	1:31:01	1:39:58	2:14:16
	Evaluar <i>test</i>	0:27:10	0:21:15	0:09:20

Tabla 8: Tiempos de entrenamiento y evaluación por split para LLM y LoRA.

C. Detalles del desarrollo del trabajo

Tarea	Tiempo (horas)
Recopilación de materiales	10
Estudio de bibliografía	30
Elaboración de resultados gráficos/numéricos	60
Redacción de la memoria	50
Total	150

Tabla 9: Tiempo aproximado de dedicación al trabajo.

Asignatura	Páginas	Descripción
Optimización II	10	Descenso del gradiente y dirección de descenso.
Análisis de datos I	15, 40-42	ACP y métricas de evaluación.
Análisis de datos II	General	Relación general con los contenidos de la asignatura.

Tabla 10: Asignaturas relacionadas con el trabajo.